Practical Exam Practice Questions

The HRMS scenario: A_1 to B_3

COS 110 | Swift Tuition | 2026 edition

These practice questions are based on the published HRMS scenario for the COS 110 Practical Exam (31 October 2026). The real exam's complete UML may differ, so read the given headers on the day. Each question has a folder in the zip file with the starter files and a makefile, laid out like the mock prac. Write only the one file the question names, build with `make`, and check the output and valgrind.

Contents

- **A_1 Exceptions**
    1. A1-01. Exceptions: Employee::setIDNum (length and gender) (100)
    2. A1-02. Exceptions: Employee::setIDNum (digits, month, day and gender) (100)
    3. A1-03. Exceptions: Employee::setIDNum (the age must match the birth year) (100)
    4. A1-04. Exceptions: Employee::extractIntFromString (100)
    5. A1-05. Exceptions: Team::operator[] throws for a bad index or an empty slot (100)
    6. A1-06. Exceptions: Department::operator[] with two error messages (100)
    7. A1-07. Exceptions: Team::addMember (NULL, already in the team, full) (100)
    8. A1-08. Exceptions: Team::assignIDs catches each error and continues (100)
    9. A1-09. Exceptions: Team::setMemberID catches two exception types (100)
    10. A1-10. Exceptions: Team::hire frees memory before it throws again (100)
- **A_2 Operator overloading**
    1. A2-01. Operator overloading: operator<< for Employee (100)
    2. A2-02. Operator overloading: operator<< for Team (skips empty slots) (100)
    3. A2-03. Operator overloading: operator<< for Department (virtual calls) (100)
    4. A2-04. Operator overloading: template operator<< for List<T> (100)
    5. A2-05. Operator overloading: Team::operator[] (const) returning const Employee& (100)
    6. A2-06. Operator overloading: Department::operator[] returning Team*& (100)
    7. A2-07. Operator overloading: Employee::operator== and operator!= (100)
    8. A2-08. Operator overloading: Department operator< and operator> by cost (100)
    9. A2-09. Operator overloading: Team::operator+= grows the array by one (100)
    10. A2-10. Operator overloading: Team::operator+ returns a merged team (100)
    11. A2-11. Operator overloading: Employee prefix and postfix operator++ (a birthday) (100)
    12. A2-12. Operator overloading: Team::operator() finds a surname (100)
- **A_3 Assignment operator**
    1. A3-01. Assignment operator: Employee::operator= (100)
    2. A3-02. Assignment operator: Team::operator= (100)
    3. A3-03. Assignment operator: Team::operator= with a compact copy (100)
    4. A3-04. Assignment operator: Department::operator= (100)
    5. A3-05. Assignment operator: Sales::operator= in a derived class (100)
    6. A3-06. Assignment operator: List<T>::operator= (100)
    7. A3-07. Copy constructor: Team::Team(const Team&) (100)
- **B_1 Recursion**
    1. B1-01. Recursion: Employee::extractIntFromString (100)
    2. B1-02. Recursion: Employee::isAllDigits (100)
    3. B1-03. Recursion: Team::countMembers (100)
    4. B1-04. Recursion: Department::totalEmployees (100)

5. B1-05. Recursion: List<T>::countNodes (100)
6. B1-06. Recursion: List<T>::printReverse (100)
7. B1-07. Recursion: List<T>::contains (100)
8. B1-08. Recursion: Company::totalCost (100)
9. B1-09. Recursion: Employee::checksum (100)
10. B1-10. Recursion: Team::oldest (100)

- **B_2 Linked lists**
  1. B2-01. Linked lists: List<T>::append (100)
  2. B2-02. Linked lists: List<T>::insertAt (100)
  3. B2-03. Linked lists: List<T>::remove by value (100)
  4. B2-04. Linked lists: List<T>::removeAt (100)
  5. B2-05. Linked lists: List<T>::reverse (100)
  6. B2-06. Linked lists: List<T>::sort by relinking (100)
  7. B2-07. Linked lists: List<T>::indexOf (100)
  8. B2-08. Linked lists: List<T>::operator[] (100)
  9. B2-09. Linked lists: List<T>::clear (100)
  10. B2-10. Linked lists: Company::removeDepartment (ownership when removing from a list) (100)
  11. B2-11. Linked lists: List<T> copy constructor (100)
- **B_3 Template specialization**
  1. B3-01. Template specialization: printItem<Department*> (100)
  2. B3-02. Template specialization: largest<Employee> (100)
  3. B3-03. Template specialization: isEqual<Department*> (100)
  4. B3-04. Template specialization: List<Department*>::printAll (100)
  5. B3-05. Template specialization: List<Department*>::clear (100)
  6. B3-06. Template specialization: totalCost<Department*> (100)
  7. B3-07. Template specialization: addition<Employee, Employee> throws (100)

A_1 Exceptions

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A1-01. Exceptions: `Employee::setIDNum` (length and gender) (100)

For this question, implement the `setIDNum` function of the `Employee` class in the file `A_1.cpp`. The function checks a new ID number. If the ID number is valid, it is stored in `idNum`. If it is not valid, the function throws an `Exception`.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void   <- implement
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}
```

Not on the published partial UML: `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

ID numbers

A South African ID number in this question is a string of 13 characters. The positions are counted from 0:

Index	Meaning
0 to 1	year of birth (YY)
2 to 3	month of birth (MM)
4 to 5	day of birth (DD)
6 to 9	gender digits: 0000 to 4999 is female (`'F'`), 5000 to 9999 is male (`'M'`)
10 to 12	other digits (not checked)

`Employee::extractIntFromString(str, start, end)` is given. It returns the number made by the characters from index `start` up to, **but not including**, index `end`. For example, `extractIntFromString("9702140123081", 6, 10)` returns `123`. It throws `Exception("Invalid range")` for a bad range and `Exception("Not a digit")` if one of those characters is not a digit.

What `setIDNum(string idNum)` must do

Do the checks in this order. Stop at the first check that fails.

1. If the length of `idNum` is not 13, throw `Exception("ID number must have 13 digits")`.
2. Read the gender digits (index 6 to 9) with `extractIntFromString(idNum, 6, 10)`. A value of 5000 or more means `'M'`. A value below 5000 means `'F'`. If this letter is not the same as the employee's `gender`, throw `Exception("ID number does not match gender")`.
3. If both checks pass, store the new ID number in `idNum`.

More rules:

- If a check fails, `idNum` must keep its old value.
- If the gender digits are not all digits, `extractIntFromString` throws `Exception("Not a digit")`. Do not catch it inside `setIDNum`. Let it go to the caller.
- `setIDNum` must not catch its own exceptions and must not print anything. The caller (for example `main`) catches them by reference, `catch (Exception& e)`, and prints `e.what()`.
- This version does not check the other characters of the ID number.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A1-02. Exceptions: `Employee::setIDNum` (digits, month, day and gender) (100)

For this question, implement the `setIDNum` function of the `Employee` class in the file `A_1.cpp`. The function checks a new ID number in several ways. If the ID number is valid, it is stored in `idNum`. If it is not valid, the function throws an `Exception` that says what is wrong.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void   <- implement
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}
```

Not on the published partial UML: `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

ID numbers

A South African ID number in this question is a string of 13 characters. The positions are counted from 0:

Index	Meaning
0 to 1	year of birth (YY)
2 to 3	month of birth (MM)
4 to 5	day of birth (DD)
6 to 9	gender digits: 0000 to 4999 is female (`'F'`), 5000 to 9999 is male (`'M'`)
10 to 12	other digits (not checked)

`Employee::extractIntFromString(str, start, end)` is given. It returns the number made by the characters from index `start` up to, **but not including**, index `end`. For example,

`extractIntFromString("9702140123081", 6, 10)` returns `123`. It throws `Exception("Invalid range")` for a bad range and `Exception("Not a digit")` if one of those characters is not a digit.

What `setIDNum(string idNum)` must do

Do the checks in this order. Stop at the first check that fails, and throw an `Exception` with exactly this message:

Order	Check	Message
1	The length is not 13.	`ID number must have 13 digits`
2	At least one character is not a digit (`'0'` to `'9'`).	`ID number must contain only digits`
3	The month (index 2 to 3) is not from 1 to 12.	`Invalid birth month`
4	The day (index 4 to 5) is not from 1 to 31.	`Invalid birth day`
5	The gender digits (index 6 to 9) do not match `gender`. 5000 or more means `'M'`, less than 5000 means `'F'`.	`ID number does not match gender`

If all five checks pass, store the new ID number in `idNum`.

More rules:

- Use the given `extractIntFromString` to read the month, the day and the gender digits.
- Check the characters one by one for check 2. Do not convert all 13 digits into one `int`: a 13-digit number does not fit in an `int`.
- Do not check how many days a month has. For example, day 30 in month 02 is accepted.
- One test ID in `main.cpp` contains the letter `O`, not the digit `0`.
- If a check fails, `idNum` keeps its old value.
- `setIDNum` must not catch its own exceptions and must not print anything. The caller catches them by reference, `catch (Exception& e)`.
- Compare an `int` with `idNum.length()` carefully: `-Wall -Werror` turns a signed/unsigned comparison warning into an error. Cast the length, for example `(int)idNum.length()`.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A1-03. Exceptions: `Employee::setIDNum` (the age must match the birth year) (100)

For this question, implement the `setIDNum` function of the `Employee` class in the file `A_1.cpp`. The function checks that a new ID number fits the employee: the birth year must give the employee's `age`, and the gender digits must give the employee's `gender`. If the ID number is valid, it is stored in `idNum`. If it is not valid, the function throws an `Exception`.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void   <- implement
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}
```

Not on the published partial UML: `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

ID numbers

A South African ID number in this question is a string of 13 characters. The positions are counted from 0:

Index	Meaning
0 to 1	year of birth (YY)
2 to 3	month of birth (MM)
4 to 5	day of birth (DD)
6 to 9	gender digits: 0000 to 4999 is female (`'F'`), 5000 to 9999 is male (`'M'`)
10 to 12	other digits (not checked)

`Employee::extractIntFromString(str, start, end)` is given. It returns the number made by the characters from index `start` up to, **but not including**, index `end`. For example, `extractIntFromString("9702140123081", 6, 10)` returns `123`. It throws `Exception("Invalid range")` for a bad range and `Exception("Not a digit")` if one of those characters is not a digit.

How to work out the age

- Read `YY` (index 0 to 1) with `extractIntFromString(idNum, 0, 2)`.
- Rule for this question only: if `YY` is 26 or less, the birth year is `2000 + YY`. Otherwise it is `1900 + YY`.
- Use 2026 as the current year. The age that the ID number gives is `2026 - birthYear`. Ignore the month and the day.

Examples: `97` gives 1997 and age 29. `26` gives 2026 and age 0. `27` gives 1927 and age 99. `00` gives 2000 and age 26.

What `setIDNum(string idNum)` must do

Do the checks in this order. Stop at the first check that fails, and throw an `Exception` with exactly this message:

Order	Check	Message
1	The length is not 13.	`ID number must have 13 digits`
2	The age from the ID number is not equal to `age`.	`ID number does not match age`
3	The gender digits (index 6 to 9) do not match `gender`. 5000 or more means `'M'`, less than 5000 means `'F'`.	`ID number does not match gender`

If all three checks pass, store the new ID number in `idNum`.

More rules:

- If a check fails, `idNum` keeps its old value.
- If the year digits or the gender digits are not digits, `extractIntFromString` throws `Exception("Not a digit")`. Do not catch it inside `setIDNum`. Let it go to the caller.
- This version does not check the month, the day or the other characters.
- `setIDNum` must not catch its own exceptions and must not print anything.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A1-04. Exceptions: `Employee::extractIntFromString` (100)

For this question, implement the static function `extractIntFromString` of the `Employee` class in the file `A_1.cpp`. The function turns a part of a string into an `int`. It throws an `Exception` when the part is not valid. The given `setIDNum` uses it to read the gender digits of an ID number.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}   <- implement
+ operator<<(ostream&, const Employee&): ostream&   {friend}
```

Not on the published partial UML: `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

What `extractIntFromString(string str, int start, int end)` must do

- The part to read starts at index `start` and stops **before** index `end`. So it is the characters `str[start]`, `str[start + 1]`, ..., `str[end - 1]`.
- Example: `extractIntFromString("9702140123081", 6, 10)` reads `"0123"` and returns `123`. Leading zeros are allowed.
- First check the range. If `start < 0`, or `start >= end`, or `end` is greater than the length of `str`, throw `Exception("Invalid range")`.
- Then check the characters in the range. If one of them is not a digit (`'0'` to `'9'`), throw `Exception("Not a digit")`. Characters outside the range are not checked.
- Otherwise return the number. A digit character `c` has the value `c - '0'`.
- A range is never longer than 9 digits, so the result always fits in an `int`.

More rules:

- Do not print anything and do not catch the exceptions. The caller catches them by reference.
- In `A_1.cpp`, write the function as `int Employee::extractIntFromString(string str, int start, int end)`. Do not repeat the word `static` outside the class.

- `str.length()` is unsigned. Cast it, `(int)str.length()`, before you compare it with an `int`, or `-Wall -Werror` stops the build.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A1-05. Exceptions: `Team::operator[]` throws for a bad index or an empty slot (100)

For this question, implement the subscript operator `operator[]` of the `Team` class in the file `A_1.cpp`. It returns the employee in a slot of the `members` array. The return type is a reference, so the operator cannot return `NULL`. When there is no employee to return, it must throw an `Exception`.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&   <- implement
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

Given code you will use

- `Team(int size)` makes an array of `size` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.

- `addMember(Employee*)` puts the employee in the first empty slot. The team owns its employees and deletes them in `~Team()`.
- `removeMember(int index)` deletes the employee in that slot and sets the slot to `NULL`. It does nothing for a bad index or an empty slot.

What `operator[](int index)` must do

1. If `index` is less than 0, or `index` is greater than or equal to `size`, throw `Exception("Index out of bounds")`.
2. Otherwise, if the slot `members[index]` is `NULL`, throw `Exception("No employee at this index")`.
3. Otherwise, return the employee in that slot: `*members[index]`.

More rules:

- Do the range check first. Reading `members[index]` with a bad index reads memory outside the array, and valgrind reports it.
- Do not catch the exception and do not print anything. The caller catches it by reference, `catch (Exception& e)`.
- In some other questions both errors use one message. Here they use two.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A1-06. Exceptions: `Department::operator[]` with two error messages (100)

For this question, implement the subscript operator `operator[]` of the `Department` class in the file `A_1.cpp`. It returns a **reference to a slot** of the `teams` array (type `Team*&`). Because it returns a reference, the caller can read the pointer and can also store a new pointer in the slot, for example `(*sales)[2] = new Team(1);`. For a bad index there is no slot to return, so the operator must throw an `Exception`.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Department
- teams: Team**
- size: int
# countEmployees(): int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&   <- implement
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}

Sales : Department
+ Sales(int)
+ getName(): string
# costToCompany(): float
```

Not on the published partial UML: `Department::countEmployees`, `Department::getSize`, `Department::getName`, `Team::addMember`, `Team::getMemberCount`, `Sales::getName`, `operator<<` (not drawn; the scenario says every class has one), the `{virtual}` markers (the UML marks nothing virtual). The real exam's complete UML may add members like these.

`Employee` is the same as in the other questions. See `Employee.h`.

Given code you will use

- `Department(int size)` makes an array of `size` team pointers. Every slot starts as `NULL`.
- `addTeam(Team*)` puts a team in the first empty slot. The department owns its teams and deletes them in `~Department()`.
- `Sales::costToCompany()` is R15000 for each employee in all its teams. This is only for the printed cost.

What `operator[](int index)` must do

1. If `index` is less than 0, throw `Exception("Negative index")`.
2. Otherwise, if `index` is greater than or equal to `size`, throw `Exception("Index too large")`.
3. Otherwise, return the slot `teams[index]` by reference.

More rules:

- An empty slot is **not** an error. If `teams[index]` is `NULL`, return the reference to that `NULL` pointer. The caller can test it (`if ((*d)[1] == NULL)`) or fill it.
- Do not catch the exception and do not print anything.
- When an assignment such as `(*d)[3] = extra;` throws, nothing is stored, so the caller still owns `extra` and must delete it (see `main.cpp`).

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A1-07. Exceptions: `Team::addMember` (NULL, already in the team, full) (100)

For this question, implement the `addMember` function of the `Team` class in the file `A_1.cpp`. It adds an employee to the `members` array. The team then owns the employee and deletes it in `~Team()`. When the employee cannot be added, the function throws an `Exception` and the team does not keep the pointer.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void   <- implement
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: `Team::addMember` (the target), `Team::removeMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

What `addMember(Employee* employee)` must do

Do the checks in this order. Stop at the first check that fails, and throw an `Exception` with exactly this message:

Order	Check	Message
1	`employee` is `NULL`.	`Employee is NULL`
2	The same pointer is already stored in **any** slot of `members`.	`Employee is already in the team`
3	There is no empty (`NULL`) slot.	`Team is full`

If all checks pass, store `employee` in the **first** empty slot (the lowest index that is `NULL`).

More rules:

- "Already in the team" means the same address (pointer), not the same name or ID number. Look at every slot, also slots after an empty slot. If the same pointer were stored twice, `~Team()` would delete it twice.
- When the function throws, it must not change `members`. The caller still owns the employee and must delete it (see `main.cpp`).
- Do not catch the exception and do not print anything.

Given code you will use

- `removeMember(int index)` deletes the employee in that slot and sets the slot to `NULL`.
- `operator[](int)` returns the employee in a slot and throws `Exception("Index out of bounds")` for a bad index or an empty slot.
- `operator<<` prints `Team:` and then each employee on its own line.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A1-08. Exceptions: `Team::assignIDs` catches each error and continues (100)

For this question, implement the `assignIDs` function of the `Team` class in the file `A_1.cpp`. The function gives ID numbers to the members of the team. `setIDNum` throws an `Exception` for a bad ID number. Your function must **catch** each exception, print its message and continue with the next slot. No exception may leave `assignIDs`.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ assignIDs(string*, int): int   <- implement
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `Team::assignIDs` (the target), `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

Given code you will use

- `Employee::setIDNum(string)` throws `Exception("ID number must have 13 digits")` if the length is not 13, `Exception("Not a digit")` if a gender digit (index 6 to 9) is not a digit, and `Exception("ID number does not match gender")` if digits 6 to 9 do not match the gender (5000 or more is `'M'`). If it throws, the old ID number is kept.
- `Team(int size)` makes an array of `size` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `addMember(Employee*)` puts the employee in the first empty slot. The team owns its employees and deletes them in `~Team()`.
- `removeMember(int index)` deletes the employee in that slot and sets the slot to `NULL`. It does nothing for a bad index or an empty slot.

What `assignIDs(string* ids, int n)` must do

`ids` is an array of `n` ID numbers. `ids[i]` is meant for the employee in slot `i` of `members`.

- If `ids` is `NULL` or `n` is 0 or less, print nothing and return 0.
- Otherwise, for each `i` from 0 to `n - 1`, print exactly one line:

Case	Line printed
`i` is not a slot of the array (`i >= size`)	`Slot <i>: no such slot`
the slot is empty (`NULL`)	`Slot <i>: empty`
`setIDNum(ids[i])` works	`Slot <i>: <firstName> <surName> -> <the new ID number>`
`setIDNum(ids[i])` throws `e`	`Slot <i>: <firstName> <surName> -> Error: <e.what()>`

- Return the number of ID numbers that were set.

Example line: `Slot 1: Sipho Dlamini -> Error: ID number does not match gender`

More rules:

- Catch by reference: `catch (Exception& e)`.
- One bad ID number must not stop the rest.
- An uncaught exception stops the program abnormally, and the mark is 0.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Hints for this practice question are in `HINTS.md` in this folder.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A1-09. Exceptions: `Team::setMemberID` catches two exception types (100)

For this question, implement the `setMemberID` function of the `Team` class in the file `A_1.cpp`. The function sets the ID number of one member and catches every exception that this can throw. Two exception types can be thrown: `InvalidIDException`, which is derived from `Exception`, and `Exception` itself. Each type must be handled in its own way.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

InvalidIDException : Exception
- badID: string
+ InvalidIDException(string, string)
+ getBadID(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ operator[](int): const Employee&
+ getMember(int): Employee*
+ getSize(): int
+ getMemberCount(): int
+ setMemberID(int, string): bool   <- implement
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: the class `InvalidIDException`, `Team::addMember`, `Team::getMember`, `Team::getMemberCount`, `Team::setMemberID` (the target), `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

Given code you will use

- `Employee::setIDNum(string idNum)`:
- throws `InvalidIDException(idNum, "ID number must have 13 digits")` if the length is not 13;
- throws `InvalidIDException(idNum, "ID number must contain only digits")` if a character is not a digit;
- throws a plain `Exception("ID number does not match gender")` if digits 6 to 9 do not match the gender (5000 or more is `'M'`);
- otherwise stores the ID number. After a throw, the old ID number is kept.
- `InvalidIDException` passes `message` to the `Exception` constructor, so `what()` returns the message. `getBadID()` returns the ID number that was rejected.
- `Team::getMember(int index)` returns the `Employee*` in a slot. It throws `Exception("No member at this index")` for a bad index or an empty slot.

What `setMemberID(int index, string idNum)` must do

Inside a `try` block, get the member with `getMember(index)` and call `setIDNum(idNum)` on it. Then print one line and return:

Case	Line printed	Return
no exception	`Member <index>: ID set to <idNum>`	`true`
an `Exception e` that is not an `InvalidIDException`	`Member <index>: <e.what()>`	`false`
an `InvalidIDException e`	`Member <index>: bad ID [<e.getBadID()>]: <e.what()>`	`false`

Example lines:

```
Member 1: bad ID [91052258000]: ID number must have 13 digits
Member 2: No member at this index
```

More rules:

- No exception may leave `setMemberID`. An uncaught exception stops the program abnormally, and the mark is 0.
- Catch each type by reference (with `&`).
- Order matters. An `InvalidIDException` is also an `Exception`. Think about which `catch` block must come first. If the order is wrong, g++ stops with an error.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.

4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Hints for this practice question are in `HINTS.md` in this folder.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A1-10. Exceptions: `Team::hire` frees memory before it throws again (100)

For this question, implement the `hire` function of the `Team` class in the file `A_1.cpp`. The function makes a new employee with `new`, gives it an ID number and adds it to the team. Two of the given functions can throw an `Exception` while `hire` runs. When that happens, your function must free the new employee (no memory leak) and then let the **same** exception reach the caller.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ hire(string, string, int, char, string): void   <- implement
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: `Team::addMember`, `Team::getMemberCount`, `Team::hire` (the target), `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

Given code you will use

- `Employee::setIDNum(string)` throws `Exception("ID number must have 13 digits")`, `Exception("Not a digit")` or `Exception("ID number does not match gender")`. (Digits 6

to 9 give the gender: 5000 or more is `'M'`.)
- `Team::addMember(Employee*)` puts the employee in the first empty slot. It throws `Exception("Team is full")` when there is no empty slot. When it throws, the team does not keep the pointer.
- The team owns the employees it holds and deletes them in `~Team()`.

What `hire(string firstName, string surName, int age, char gender, string idNum)` must do

1. Make a new employee with `new Employee(firstName, surName, age, gender)`.
2. Call `setIDNum(idNum)` on the new employee.
3. Call `addMember` with the new employee.

If step 2 or step 3 throws an `Exception`, `hire` must free the new employee and then throw the same exception, so that the caller gets the same message. If nothing throws, the team owns the new employee, and `hire` must not delete it.

More rules:

- Do the steps in this order. When the ID number is bad **and** the team is full, the caller must get the ID number message.
- `hire` must not print anything.
- valgrind must report no leaks, also after many failed hires.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_1` folder.
2. Run `make A_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Hints for this practice question are in `HINTS.md` in this folder.

A_2 Operator overloading

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-01. Operator overloading: `operator<<` for `Employee` (100)

For this question, implement the stream insertion operator `operator<<` for the `Employee` class in the file `A_2.cpp`. It is a `friend` function of `Employee`, so it can read the private members.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}   <- implement
```

Not on the published partial UML: `operator<<` (not drawn; the scenario says every class has one; the one for `Employee` is the target), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

What `operator<<(ostream& out, const Employee& e)` must print

```
<firstName> <surName>, <age>, <gender>, ID: <idNum>
```

- If `idNum` is the empty string `""` (no ID number set yet), print `none` in its place.
- Examples: `Thandi Mokoena, 29, F, ID: 9702140123081` and `Sipho Dlamini, 35, M, ID: none`.
- There is one space after each comma and after `ID:`. Nothing is printed after the ID number: **no newline**. `main` adds `endl` itself.
- Write to the stream `out`, not to `cout`. The marking program may send the output to a different stream.
- Return `out`, so that calls can be chained: `cout << a << " | " << b << endl;`
- Write the word `friend` only in the class. Do not write it again in `A_2.cpp`.

Note: `e` is a `const` reference, and the getters (`getFirstName()` and so on) are not `const` member functions. A `const` object cannot call them. Use the data members directly (`e.firstName`), which a friend may do.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-02. Operator overloading: `operator<<` for `Team` (skips empty slots) (100)

For this question, implement the stream insertion operator `operator<<` for the `Team` class in the file `A_2.cpp`. It is a `friend` function of `Team`. It prints a heading and then every employee in the team.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}   <- implement
```

Not on the published partial UML: `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one; the one for `Team` is the target), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

Given code you will use

- `Team(int size)` makes an array of `size` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `addMember(Employee*)` puts the employee in the first empty slot. The team owns its employees and deletes them in `~Team()`.

- `removeMember(int index)` deletes the employee in that slot and sets the slot to `NULL`. It does nothing for a bad index or an empty slot.
- `Employee`'s `operator<<` (given) prints `<firstName> <surName>, <age>, <gender>, ID: <idNum>`, or `ID: none` when no ID number is set. It does not print a newline.

What `operator<<(ostream& out, const Team& t)` must print

1. A heading line: `Team (<count>/<size>):` where `<count>` is the number of slots that are not `NULL`, and `<size>` is the length of the array.
2. If `<count>` is 0, one more line: two spaces, then `(no members)`.
3. Otherwise, for each slot that is not `NULL`, in slot order (index 0 first): two spaces, then the employee printed with `Employee`'s `operator<<`.

Every line ends with `endl`, including the last one. Example for a team of size 3 with an empty slot 0:

```
Team (1/3):
  Sipho Dlamini, 35, M, ID: none
```

More rules:

- Skip empty (`NULL`) slots. Do not dereference them.
- Write to `out`, not to `cout`, and return `out`.
- Note: `t` is a `const` reference. `getMemberCount()` and `operator[]` are not `const` member functions, so `t` cannot call them. Count the members yourself using `t.members` and `t.size`, which a friend may read.
- Write the word `friend` only in the class. Do not write it again in `A_2.cpp`.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-03. Operator overloading: `operator<<` for `Department` (virtual calls) (100)

For this question, implement the stream insertion operator `operator<<` for the `Department` class in the file `A_2.cpp`. It is a `friend` function of `Department`. The same operator must print every kind of department (Sales, Marketing, HR and Development) correctly.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Department
- teams: Team**
- size: int
# countEmployees(): int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}   <- implement

Sales : Department
+ Sales(int)
+ getName(): string
# costToCompany(): float

Marketing : Department
+ Marketing(int)
+ getName(): string
# costToCompany(): float

HR : Department
+ HR(int)
+ getName(): string
# costToCompany(): float

Development : Department
+ Development(int)
+ getName(): string
# costToCompany(): float

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ operator[](int): const Employee&
+ getSize(): int
```

```
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: `Department::countEmployees`, `Department::getSize`, `Department::getName`, `Sales::getName`, `Marketing::getName`, `HR::getName`, `Development::getName`, `Team::addMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one; the one for `Department` is the target), the `{virtual}` markers (the UML marks nothing virtual). The real exam's complete UML may add members like these.

`Employee` is the same as in the other questions. See `Employee.h`.

Given code you will use

- `costToCompany()` is `virtual`. The base `Department` returns 0. Each subclass returns its number of employees (in all its teams) times a monthly cost: Sales R15000, Marketing R18000, HR R12000, Development R25000. `getName()` is also `virtual` and returns the class name, for example `"Sales"`.
- `Team`'s `operator<<` (given) prints `Team (<count>/<size>):` on one line, then each employee on its own line with two spaces in front, or the line `(no members)` for a team with no employees. Every line ends with `endl`.

What `operator<<(ostream& out, Department& d)` must print

1. A heading line: `Department: <name> (cost R<cost>)`, where `<name>` is `d.getName()` and `<cost>` is `d.costToCompany()`. End the line with `endl`.
2. Each team that is not `NULL`, in slot order, printed with `Team`'s `operator<<`.
3. If the department has no team at all (every slot is `NULL`, or the size is 0), print one line instead (two spaces, then `(no teams)`) and end it with `endl`.

Example:

```
Department: Development (cost R50000)
Team (2/2):
  Thandi Mokoena, 29, F, ID: none
  Sipho Dlamini, 35, M, ID: none
```

More rules:

- Print the cost with the normal `cout` format, as `R50000` (no decimals). All costs in this question stay below one million.
- Friend functions are not inherited, so there is no `operator<<` for `Sales`. A `Sales` object is printed through this operator as a `Department&`. Call the `virtual` functions `getName()` and `costToCompany()` so that the subclass versions run.
- The parameter is `Department&` (not `const`), because `getName()` and `costToCompany()` are not `const` member functions.
- Write to `out`, not to `cout`, and return `out`.
- Write the word `friend` only in the class. Do not write it again in `A_2.cpp`.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-04. Operator overloading: template `operator<<` for `List<T>` (100)

For this question, implement the stream insertion operator `operator<<` for the template class `List<T>` in the file `A_2.cpp`. The operator is a template function and a `friend` of `List<T>`.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ operator<<(ostream&, const List<T>&): ostream&   {friend}   <- implement

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `operator<<` (not drawn; the scenario says every class has one; the one for `List<T>` is the target). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

Given code you will use

- `prepend(T value)` puts a new node holding a copy of `value` at the **front** of the list. So after `prepend(3)`, `prepend(2)`, `prepend(1)` the list is 1, 2, 3.
- `getData()` returns the value in a node. `next()` returns the pointer to the next node (`NULL` after the last node).
- `List.h` already declares the operator before the class and makes it a friend with `operator<< <>`:

```
template <class T>
ostream& operator<<(ostream& out, const List<T>& list);
...
friend ostream& operator<< <>(ostream& out, const List<T>& list);
```

What `operator<<(ostream& out, const List<T>& list)` must print

- If the list is empty, print `Empty List`.
- Otherwise print every value from the head to the last node, with `->` between two values. Example: `1->2->3`. Print each value with its own `operator<<`.
- Print no `->` after the last value, and **no newline**. `main` adds `endl` itself.
- Write to `out`, not to `cout`, and return `out`.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `A_2.cpp` at the bottom (the COS 110 rule for templates).
- `A_2.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef A_2_CPP`, `#define A_2_CPP`, `#endif`) makes both ways work. Keep your code between `#define A_2_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `A_2.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Write the operator as a function template. Its first two lines are:

```
template <class T>
ostream& operator<<(ostream& out, const List<T>& list) {
```

- Keep the `#include "List.h"` line in `A_2.cpp`.
- Do not write `<>` in the definition. The `<>` belongs only to the `friend` line in `List.h`.
- Write the word `friend` only in the class. Do not write it again in `A_2.cpp`.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-05. Operator overloading: `Team::operator[]` (const) returning `const Employee&` (100)

For this question, implement the subscript operator `operator[]` of the `Team` class in the file `A_2.cpp`. Like an array, `team[i]` gives the employee in slot `i`. In this question the operator is a `const` member function, so it also works on a `const Team&`.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&   {const}   <- implement
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one), the `{const}` on `Team::operator[]` (the published UML does not mark it `const`), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

The UML marks this with `{const}`. In C++ the word `const` goes after the parameter list: `const Employee& operator[](int index) const;`.

Given code you will use

- `Team(int size)` makes an array of `size` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `addMember(Employee*)` puts the employee in the first empty slot. The team owns its employees and deletes them in `~Team()`.
- `removeMember(int index)` deletes the employee in that slot and sets the slot to `NULL`. It does nothing for a bad index or an empty slot.

What `operator[](int index) const` must do

1. If `index` is less than 0, or greater than or equal to `size`, or the slot `members[index]` is `NULL`, throw `Exception("Index out of bounds")`.
2. Otherwise, return the employee in that slot by reference: `*members[index]`.

More rules:

- The return type is a reference, so the operator cannot return `NULL`. Throwing is the only way to report a bad index.
- Check the range before you read `members[index]`.
- Write the word `const` after the parameter list in `A_2.cpp` as well, exactly as in the header. A `const` member function cannot change the team.
- Do not print anything and do not catch the exception.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-06. Operator overloading: `Department::operator[]` returning `Team*&` (100)

For this question, implement the subscript operator `operator[]` of the `Department` class in the file `A_2.cpp`. The return type is `Team*&`: a **reference to a pointer** in the `teams` array. The caller can use it to read the pointer, to call a function on the team, or to store a new pointer in the slot:

```
(*dev)[0] = new Team(2);                  // store a pointer in slot 0
(*dev)[0]->addMember(new Employee(...));  // use the team in slot 0
Team* temp = (*dev)[0];                   // read the pointer in slot 0
```

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Department
- teams: Team**
- size: int
# countEmployees(): int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&   <- implement
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}

Development : Department
+ Development(int)
+ getName(): string
# costToCompany(): float

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: `Department::countEmployees`, `Department::getSize`, `Department::getName`, `Development::getName`, `Team::addMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one), the `{virtual}` markers (the UML marks nothing virtual). The real exam's complete UML may add members like these.

`Employee` is the same as in the other questions. See `Employee.h`.

Given code you will use

- `Department(int size)` makes an array of `size` team pointers. Every slot starts as `NULL`.
- `addTeam(Team*)` puts a team in the first `NULL` slot and throws `Exception("Department is full")` when there is none.
- The department owns its teams and deletes every slot in `~Department()`.
- `operator<<` prints `<name> (cost R<cost>)` and then each team that is not `NULL`. Development costs R25000 per employee.

What `operator[](int index)` must do

1. If `index` is less than 0, or greater than or equal to `size`, throw `Exception("Index out of bounds")`.
2. Otherwise, return the slot `teams[index]` **by reference**.

More rules:

- An empty slot is not an error. Return the reference to the `NULL` pointer, so that the caller can test it or fill it.
- Return the array element itself, not a local copy of the pointer. A reference to a local variable is not valid after the function returns, and `-Wall -Werror` rejects it.
- Do not print anything and do not catch the exception.
- The caller owns the team in a slot. Before it stores a new pointer in a slot that already holds a team, the caller must delete the old team.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-07. Operator overloading: `Employee::operator==` and `operator!=` (100)

For this question, implement the two comparison operators `operator==` and `operator!=` of the `Employee` class in the file `A_2.cpp`. Two `Employee` objects are the same person when they have the same ID number.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator==(const Employee&): bool   {const}   <- implement
+ operator!=(const Employee&): bool   {const}   <- implement
+ operator<<(ostream&, const Employee&): ostream&   {friend}
```

Not on the published partial UML: `Employee::operator==` (the target), `Employee::operator!=` (the target), `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

Both operators are `const` member functions (see `Employee.h`).

What the operators must return

`a == b` is `true` when the two `idNum` strings are equal. Otherwise it is `false`. The names, the age and the gender are **not** compared.

`a != b` must always give the opposite of `a == b`.

Results to check your work:

Case	`a == b`	`a != b`
same ID number, different names	`true`	`false`

Case	`a == b`	`a != b`
different ID numbers	`false`	`true`
one ID number set, the other not set	`false`	`true`
both ID numbers not set	`true`	`false`
an employee compared with itself	`true`	`false`

Look at row 4 carefully (both ID numbers not set). An ID number that is not set yet is the empty string `""`. Two employees with no ID number both have `""`, so they are equal, even if their names are different. The marking program checks this case.

More rules:

- Inside the member function you may read `other.idNum` directly, because `other` is an object of the same class.
- Write `const` after the parameter list in `A_2.cpp`, exactly as in the header.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Hints for this practice question are in `HINTS.md` in this folder.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-08. Operator overloading: `Department` `operator<` and `operator>` by cost (100)

For this question, implement the two comparison operators `operator<` and `operator>` of the `Department` class in the file `A_2.cpp`. Departments are compared by how much they cost the company each month.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Department
- teams: Team**
- size: int
# countEmployees(): int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<(Department&): bool   <- implement
+ operator>(Department&): bool   <- implement
+ operator<<(ostream&, Department&): ostream&   {friend}

Sales : Department
+ Sales(int)
+ getName(): string
# costToCompany(): float

Marketing : Department
+ Marketing(int)
+ getName(): string
# costToCompany(): float

HR : Department
+ HR(int)
+ getName(): string
# costToCompany(): float

Development : Department
+ Development(int)
+ getName(): string
# costToCompany(): float

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
```

```
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: `Department::countEmployees`, `Department::getSize`, `Department::getName`, `Department::operator<` (the target), `Department::operator>` (the target), `Sales::getName`, `Marketing::getName`, `HR::getName`, `Development::getName`, `Team::addMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one), the `{virtual}` markers (the UML marks nothing virtual). The real exam's complete UML may add members like these.

`Employee` is the same as in the other questions. See `Employee.h`.

Given code you will use

- `costToCompany()` is `virtual`. The base `Department` returns 0. Each subclass returns its number of employees (in all its teams) times a monthly cost: Sales R15000, Marketing R18000, HR R12000, Development R25000. `getName()` is also `virtual` and returns the class name, for example `"Sales"`.
- `addTeam(Team*)` puts a team in the first empty slot. `countEmployees()` (protected) counts the employees in all teams.

What the operators must return

- `a < b` is `true` when `a.costToCompany()` is less than `b.costToCompany()`. Otherwise it is `false`.
- `a > b` is `true` when `a.costToCompany()` is greater than `b.costToCompany()`. Otherwise it is `false`.
- When the two costs are equal, both operators return `false`.

Example: a Sales department with 2 employees costs R30000, and a Development department with 1 employee costs R25000. So `sales < dev` is `false` and `sales > dev` is `true`.

More rules:

- Compare the **cost**, not the number of employees. Different departments have different costs per employee.
- Call `costToCompany()` on both objects. It is `virtual`, so the subclass rule is used even through a `Department&`.
- The parameter is `Department&` (not `const`), because `costToCompany()` is not a `const` member function.
- Do not print anything.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.

3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-09. Operator overloading: `Team::operator+=` grows the array by one (100)

For this question, implement the compound assignment operator `operator+=` of the `Team` class in the file `A_2.cpp`. `team += employee;` adds a **copy** of the employee to the team. The `members` array always grows by one slot, so the team is never full.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator+=(Employee*): Team&   <- implement
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: `Team::addMember`, `Team::getMemberCount`, `Team::operator+=` (the target), `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

Given code you will use

- `Team(int size)` makes an array of `size` pointers, all `NULL`. The team owns its employees and deletes them in `~Team()`.
- `addMember(Employee*)` puts an employee in the first `NULL` slot (it does not grow the array).

- `getSize()` returns `size`. `operator<<` prints `Team (<count>/<size>):` and then each employee.
- `Employee(const Employee&)` is the copy constructor.

What `operator+=(Employee* employee)` must do

1. If `employee` is `NULL`, change nothing and return `*this`.
2. Otherwise:
3. make a new array of `size + 1` pointers;
4. copy every pointer from `members` into the same index of the new array. Empty (`NULL`) slots stay `NULL` and keep their places;
5. put a **new copy** of the employee, `new Employee(*employee)`, in the last slot (index `size`);
6. delete the old array (only the array, not the employees in it);
7. let `members` point to the new array and add 1 to `size`.
8. Return `*this` by reference, so that calls can be chained: `(team += a) += b;`

More rules:

- The team stores its own copy. The caller still owns the employee it passed in, and deletes it later. A change to the caller's employee afterwards does not change the team.
- Always add at the end. Do not use an empty slot in the middle.
- valgrind must report no leaks and no double deletes. If the old array is not deleted, the output is still correct, but valgrind reports a leak.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-10. Operator overloading: `Team::operator+` returns a merged team (100)

For this question, implement the addition operator `operator+` of the `Team` class in the file `A_2.cpp`. `a + b` makes a **new** team that holds copies of the members of `a` and then copies of the members of `b`. Neither `a` nor `b` changes.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ Team(const Team&)
+ operator=(const Team&): Team&
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator+(const Team&): Team   {const}   <- implement
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: `Team(const Team&)`, `Team::operator=`, `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `Team::operator+` (the target), `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

Given code you will use

- `Team(int size)` makes an array of `size` pointers, all `NULL`. The team owns its employees and deletes them in `~Team()`.
- `Team` has a destructor, a copy constructor and an assignment operator that copies deeply (the rule of three). A deep copy gets its own new `Employee` objects; it does not share them. All three are given: `~Team()`, `Team(const Team& other)` and `Team& operator=(const Team& other)`. So both `Team c = a + b;` (copy constructor) and `c = a + b;` on a team that already exists (assignment operator) are safe.
- `removeMember(int index)` deletes the employee in that slot and sets the slot to `NULL`.
- `Employee(const Employee&)` is the copy constructor of `Employee`.
- `operator<<` prints `Team (<count>/<size>):` and then each employee.

What `operator+(const Team& other) const` must do

1. Count the members (slots that are not `NULL`) in this team and in `other`. Call the total `count`.
2. Make a new team with exactly `count` slots.
3. Fill it in this order, with no gaps: first a copy of each member of this team (in slot order), then a copy of each member of `other` (in slot order). Skip `NULL` slots. Each copy is made with `new Employee(*...)`.
4. Return the new team **by value**.

Example: this team is `[Thandi, Sipho, NULL]` and `other` is `[Anna, NULL]`. The result has size 3: `[Thandi, Sipho, Anna]`.

More rules:

- The new team must own its own copies. Do not copy the pointers, or two teams would delete the same employee.
- If both teams have no members, the result is a team of size 0.
- `a + a` is allowed and gives every member twice.
- The function is `const`: it must not change this team. Write `const` after the parameter list in `A_2.cpp` too.
- Inside the function you may read `other.members` and `other.size`, and you may set the members of the new team directly, because they are objects of the same class.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-11. Operator overloading: `Employee` prefix and postfix `operator++` (a birthday) (100)

For this question, implement **both** increment operators of the `Employee` class in the file `A_2.cpp`. Each one is a birthday: it adds 1 to `age`. The two versions return different things.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator++(): Employee&   <- implement
+ operator++(int): Employee   <- implement
+ operator<<(ostream&, const Employee&): ostream&   {friend}
```

Not on the published partial UML: `Employee::operator++` (the target), `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

What the operators must do

Operator	Use	What it does	Returns
`Employee& operator++()`	`++e` (prefix)	adds 1 to `age`	the employee itself, `*this`, by reference
`Employee operator++(int)`	`e++` (postfix)	adds 1 to `age`	a **copy** of the employee from **before** the change, by value

Example: if `e` is 30, then `Employee old = e++;` makes `old` 30 and `e` 31.

More rules:

- Nothing else changes: not the names, not the gender and not the ID number.

- The `int` parameter of the postfix version has no name and is not used. It only tells C++ which version is meant.
- Because prefix returns a reference to the same object, `++(++e)` adds 2 to `e`.
- Use the given copy constructor `Employee(const Employee&)` to make the copy in the postfix version.
- `Employee`'s `operator<<` (given) prints `<firstName> <surName>, <age>, <gender>, ID: <idNum>`, or `ID: none` when no ID number is set. It does not print a newline.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A2-12. Operator overloading: `Team::operator()` finds a surname (100)

For this question, implement the function call operator `operator()` of the `Team` class in the file `A_2.cpp`. With it, a team object can be "called" like a function: `team("Dlamini")` gives the index of the member whose surname is `Dlamini`.

UML (only the classes this question needs)

```
Exception
+ message: string
+ Exception(string)
+ what(): string

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator()(string): int   {const}   <- implement
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

Not on the published partial UML: `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `Team::operator()` (the target), `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

`operator()` is a `const` member function (see `Team.h`).

Given code you will use

- `Team(int size)` makes an array of `size` pointers. Every slot starts as `NULL`. An empty slot is `NULL`.
- `addMember(Employee*)` puts the employee in the first empty slot. The team owns its employees and deletes them in `~Team()`.
- `removeMember(int index)` deletes the employee in that slot and sets the slot to `NULL`. It does nothing for a bad index or an empty slot.

What `operator()(string surName) const` must return

- Look at the slots in order, from index 0 to `size - 1`. Skip empty (`NULL`) slots.
- Return the index of the **first** member whose surname is equal to `surName`.
- If no member has that surname, return `-1`.

More rules:

- The comparison is exact: capital letters and spaces matter. `"nkosi"` does not match `"Nkosi"`.
- Only surnames are compared, not first names.
- A team with no members, or of size 0, always gives `-1`.
- `getSurName()` is not a `const` function, but you can call it as `members[i]->getSurName()`. In a `const` function the pointer cannot change, but the `Employee` it points to can.
- Write `const` after the parameter list in `A_2.cpp` too. Do not print anything.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_2` folder.
2. Run `make A_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

A_3 Assignment operator

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A3-01. Assignment operator: `Employee::operator=` (100)

For this question, implement the assignment operator of the `Employee` class in the file `A_3.cpp`. All the other functions of `Employee` and `Exception` are given in the other files.

UML (only the classes this question needs)

```
Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ operator=(const Employee&): Employee&   <- implement
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Exception
+ message: string
+ Exception(string)
+ what(): string
```

Not on the published partial UML: `Employee::operator=` (the target), `operator<<` (not drawn; the scenario says every class has one), the `int` return type of `extractIntFromString` (the UML shows no return type). The real exam's complete UML may add members like these.

What `operator=` must do

The statement `a = b;` makes the employee `a` hold the same details as the employee `b`. Write the four steps of an assignment operator:

1. If the object is assigned to itself (`a = a;`), do nothing and return the object.
2. Free the old memory of this object. `Employee` has no pointers (its attributes are `string`, `int` and `char`), so there is nothing to delete in this step.
3. Copy every attribute from the other employee: `firstName`, `surName`, `age`, `gender` and `idNum`. An empty `idNum` (`""`) is copied too.
4. Return the current object by reference (`*this`), so that chaining works: `a = b = c;` first does `b = c`, then `a = b`.

The given `operator<<` in `Employee.cpp` prints `<firstName> <surName>, <age>, <gender>, ID: <idNum>`, and `none` when `idNum` is empty.

Rules

- Implement only `Employee::operator=` in `A_3.cpp`. The declaration is already in `Employee.h`.
- The other employee is passed as `const Employee&`. The getters are not `const` functions, so you cannot call them on `other`. Use the attributes directly (`other.firstName` and so on). This is allowed inside the class's own operator.
- The marking program checks every attribute, self-assignment (`a = a;`) and chaining (`a = b = c;`).
- Your code must compile with `-std=c++98 -Wall -Werror`. Memory is marked: `valgrind` must show no leaks and no invalid reads or frees.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_3` folder.
2. Run `make A_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A3-02. Assignment operator: `Team::operator=` (100)

For this question, implement the assignment operator of the `Team` class in the file `A_3.cpp`. All the other functions of `Team`, `Employee` and `Exception` are given in the other files.

UML (only the classes this question needs)

```
Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ operator=(const Team&): Team&   <- implement
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Exception
+ message: string
+ Exception(string)
+ what(): string
```

The other `Employee` members are in `Employee.h`.

Not on the published partial UML: `Team::operator=` (the target), `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

How the given `Team` works:

- `members` is an array of `size` pointers. Each slot holds an `Employee*` or `NULL`. `NULL` means the slot is empty.
- The team owns its employees. `~Team()` deletes every employee and then the array.
- `addMember` puts an employee in the first empty slot. It throws `Exception("Team is full")` when there is no empty slot.
- `removeMember(i)` deletes the employee in slot `i` and sets the slot to `NULL`.
- The `Team` copy constructor is not used in this question. `Team.h` declares it private, so a `Team` cannot be copied by mistake.

What `operator=` must do

The statement `a = b;` makes team `a` a deep copy of team `b`. A deep copy gets its own new objects; it does not share them with the original.

1. If the team is assigned to itself (`a = a;`), do nothing and return the team.
2. Delete every employee that this team owns, and then delete the `members` array.
3. Set `size` to the other team's size and create a new array of that size.
4. For every slot of the other team: if the slot is `NULL`, this slot is `NULL`. Otherwise, this slot gets a **new copy** of that employee (use the `Employee` copy constructor). Every employee stays in the same slot number.
5. Return the current object by reference (`*this`), so that `a = b = c;` works.

After the assignment, the two teams must not share any `Employee` object. A change to one team must not show in the other team, and deleting both teams must not delete any employee twice.

Rules

- Implement only `Team::operator=` in `A_3.cpp`. The declaration is already in `Team.h`.
- `other` is a `const Team&`. `getSize()` and `getMemberCount()` are not `const` functions, so use `other.size` and `other.members` directly. This is allowed inside the class's own operator.
- Your code must compile with `-std=c++98 -Wall -Werror`. Memory is marked: `valgrind` must show no leaks and no invalid reads or frees.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_3` folder.
2. Run `make A_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A3-03. Assignment operator: `Team::operator=` with a compact copy (100)

For this question, implement the assignment operator of the `Team` class in the file `A_3.cpp`. All the other functions of `Team`, `Employee` and `Exception` are given in the other files.

Note: in this question the copy is **compact**. The new team has no empty slots. This rule is different from a normal deep copy, so read the steps carefully.

UML (only the classes this question needs)

```
Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ operator=(const Team&): Team&   <- implement
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Exception
+ message: string
+ Exception(string)
+ what(): string
```

The other `Employee` members are in `Employee.h`.

Not on the published partial UML: `Team::operator=` (the target), `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

How the given `Team` works:

- `members` is an array of `size` pointers. Each slot holds an `Employee*` or `NULL`. `NULL` means the slot is empty.
- The team owns its employees. `~Team()` deletes every employee and then the array.
- `addMember` puts an employee in the first empty slot. It throws `Exception("Team is full")` when there is no empty slot.
- `removeMember(i)` deletes the employee in slot `i` and sets the slot to `NULL`.

- The `Team` copy constructor is not used in this question. `Team.h` declares it private, so a `Team` cannot be copied by mistake.

What `operator=` must do

The statement `a = b;` makes team `a` a compact deep copy of team `b`. A deep copy gets its own new objects; it does not share them with the original.

1. If the team is assigned to itself (`a = a;`), do nothing and return the team. The empty slots stay as they are.
2. Delete every employee that this team owns, and then delete the `members` array.
3. Count the employees (the slots that are not `NULL`) in the other team. This number is the new `size`. Create a new array of that size. If the other team has no employees, the new size is `0`.
4. Copy the employees of the other team into the new array, from slot `0` upwards, in the same order as in the other team. Each slot gets a **new copy** of the employee (use the `Employee` copy constructor). The new array has no `NULL` slots.
5. Return the current object by reference (`*this`), so that `a = b = c;` works.

Example: if `b` has size 4 with slots `Sipho, empty, Ravi, empty`, then after `a = b;` team `a` has size 2 with slots `Sipho, Ravi`.

After the assignment, the two teams must not share any `Employee` object.

Rules

- Implement only `Team::operator=` in `A_3.cpp`. The declaration is already in `Team.h`.
- `other` is a `const Team&`. `getSize()` and `getMemberCount()` are not `const` functions, so use `other.size` and `other.members` directly. This is allowed inside the class's own operator.
- Your code must compile with `-std=c++98 -Wall -Werror`. Memory is marked: `valgrind` must show no leaks and no invalid reads or frees.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_3` folder.
2. Run `make A_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A3-04. Assignment operator: `Department::operator=` (100)

For this question, implement the assignment operator of the `Department` class in the file `A_3.cpp`. All the other functions of `Department`, `Team`, `Employee` and `Exception` are given in the other files.

UML (only the classes this question needs)

```
Department
- teams: Team**
- size: int
+ Department(int)
+ ~Department()   {virtual}
+ operator=(const Department&): Department&   <- implement
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ Team(const Team&)
+ ~Team()
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Exception
+ message: string
+ Exception(string)
+ what(): string
```

The other `Employee` members are in `Employee.h`.

Not on the published partial UML: `Department::operator=` (the target), `Department::getSize`, `Department::getName`, the `{virtual}` markers (the UML marks nothing virtual), `Team(const Team&)`, `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

How the given classes work:

- `teams` is an array of `size` pointers. Each slot holds a `Team*` or `NULL`. `NULL` means the slot is empty.
- The department owns its teams. `~Department()` deletes every team and then the array. Each team deletes its own employees.
- `Team(const Team&)` is given. It makes a deep copy of a team: a new array, a new copy of every employee, and `NULL` slots stay `NULL`.
- `operator[](int)` returns a reference to a slot. It throws `Exception("Index out of bounds")` for an index outside `0` to `size - 1`.
- `addTeam` puts a team in the first empty slot and throws `Exception("Department is full")` when there is none.
- In `Department`, `costToCompany()` returns `0` and `getName()` returns `"Department"`.
- Two members are not used in this question, so they are declared private: the `Department` copy constructor and `Team::operator=`.

What `operator=` must do

The statement `a = b;` makes department `a` a deep copy of department `b`. A deep copy gets its own new objects; it does not share them with the original.

1. If the department is assigned to itself (`a = a;`), do nothing and return the department.
2. Delete every team that this department owns, and then delete the `teams` array.
3. Set `size` to the other department's size and create a new array of that size.
4. For every slot of the other department: if the slot is `NULL`, this slot is `NULL`. Otherwise, this slot gets a **new copy** of that team (use the `Team` copy constructor). Every team stays in the same slot number. A team with no employees is still a team, so it is copied too.
5. Return the current object by reference (`*this`), so that `a = b = c;` works.

After the assignment, the two departments must not share any `Team` or `Employee` object.

Rules

- Implement only `Department::operator=` in `A_3.cpp`. The declaration is already in `Department.h`.
- `other` is a `const Department&`. Use `other.size` and `other.teams` directly. This is allowed inside the class's own operator.
- Your code must compile with `-std=c++98 -Wall -Werror`. Memory is marked: `valgrind` must show no leaks and no invalid reads or frees.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_3` folder.
2. Run `make A_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A3-05. Assignment operator: `Sales::operator=` in a derived class (100)

For this question, implement the assignment operator of the `Sales` class in the file `A_3.cpp`. `Sales` inherits from `Department`. All the other functions are given in the other files, including `Department::operator=`.

Note: in this question `Sales` has one extra attribute, `target`.

UML (only the classes this question needs)

```
Department
- teams: Team**
- size: int
# countEmployees(): int
+ Department(int)
+ ~Department()   {virtual}
+ operator=(const Department&): Department&
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}

Sales : Department
- target: int
+ Sales(int)
+ operator=(const Sales&): Sales&   <- implement
+ setTarget(int): void
+ getTarget(): int
+ getName(): string
# costToCompany(): float

Team
- members: Employee**
- size: int
+ Team(int)
+ Team(const Team&)
+ ~Team()
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
```

```
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Exception
+ message: string
+ Exception(string)
+ what(): string
```

The other `Employee` members are in `Employee.h`.

Not on the published partial UML: `Sales::operator=` (the target), `Sales::target`, `Sales::setTarget`, `Sales::getTarget`, `getName()` in `Department` and `Sales`, `Department::operator=`, `Department::countEmployees`, `Department::getSize`, the `{virtual}` markers (the UML marks nothing virtual), `Team(const Team&)`, `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

How the given classes work:

- `Department::operator=` is given. It makes a deep copy of the other department's `teams` array and sets `size`. A deep copy gets its own new objects; it does not share them with the original. It checks for self-assignment and returns `*this`.
- `teams` and `size` are **private** in `Department`, so `Sales` cannot use them directly.
- A new `Sales` object has `target` equal to `0`. `Sales::costToCompany()` returns the number of employees times 15000.
- The `Department` copy constructor and `Team::operator=` are not used in this question, so they are declared private.

What `Sales::operator=` must do

The statement `a = b;` (both are `Sales` objects) makes `a` a deep copy of `b`.

1. If the object is assigned to itself (`a = a;`), do nothing and return the object.
2. Use the given `Department::operator=` so that `Department` copies its own part. `teams` and `size` are private in `Department`, so `Sales` cannot copy them itself.
3. Copy `target` from the other object.
4. Return the current object by reference (`*this`), so that `a = b = c;` works.

Rules

- Implement only `Sales::operator=` in `A_3.cpp`. The declaration is already in `Sales.h`.
- Your code must compile with `-std=c++98 -Wall -Werror`. Memory is marked: `valgrind` must show no leaks and no invalid reads or frees.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_3` folder.
2. Run `make A_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.

4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Hints for this practice question are in `HINTS.md` in this folder.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A3-06. Assignment operator: `List<T>::operator=` (100)

For this question, implement the assignment operator of the class template `List<T>` in the file `A_3.cpp`. All the other functions of `List<T>` and `Node<T>` are given in the other files.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ operator=(const List<T>&): List<T>&   <- implement
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `List<T>::operator=` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

How the given classes work:

- A node owns its `data`. `~Node()` deletes `data`. It does **not** delete the next node.
- `prepend(value)` makes a new node holding `new T(value)` and puts it at the front of the list. It adds 1 to `size`.
- `~List()` deletes every node.
- `next()` returns a reference to the node's `nextNode` pointer, so `curr->next() = newNode;` links a new node after `curr`.
- `operator<<` prints `Empty List` for an empty list, or the values joined by `->`, for example `1->2->3`.
- The `List` copy constructor and the `Node` copy constructor and `operator=` are not used in this question, so they are declared private.

What `operator=` must do

The statement `a = b;` makes list `a` a deep copy of list `b`. A deep copy gets its own new nodes and data; it does not share them with the original.

1. If the list is assigned to itself (`a = a;`), do nothing and return the list.
2. Delete every node of this list. Set `headNode` to `NULL` and `size` to `0`.
3. Copy every node of the other list, **in the same order**. Each new node holds its own `new T(...)` copy of the value. (Note: `prepend` adds at the front, so using it while you walk from the head would

reverse the order.)
4. After the copy, `size` is the same as the other list's size.
5. Return the current object by reference (`*this`), so that `a = b = c;` works.

After the assignment, the two lists must not share any node or any data.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `A_3.cpp` at the bottom (the COS 110 rule for templates).
- `A_3.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef A_3_CPP`, `#define A_3_CPP`, `#endif`) makes both ways work. Keep your code between `#define A_3_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `A_3.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Your function is a template member. It starts like this:

```
template <class T>
List<T>& List<T>::operator=(const List<T>& other) {
```

Rules

- Implement only `List<T>::operator=` in `A_3.cpp`. The declaration is already in `List.h`.
- `other` is a `const List<T>&`. Use `other.headNode` directly. This is allowed inside the class's own operator.
- Your code must compile with `-std=c++98 -Wall -Werror`. Memory is marked: `valgrind` must show no leaks and no invalid reads or frees.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_3` folder.
2. Run `make A_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

A3-07. Copy constructor: `Team::Team(const Team&)` (100)

For this question, implement the copy constructor of the `Team` class in the file `A_3.cpp`. All the other functions of `Team`, `Employee` and `Exception` are given in the other files, including `Team::operator=`.

The exam's A_3 question asks for an assignment operator. The copy constructor is its partner in the rule of three (a destructor, a copy constructor and an assignment operator that copies deeply), and it is often written by reusing `operator=`, so it is practised in this section.

UML (only the classes this question needs)

```
Team
- members: Employee**
- size: int
+ Team(int)
+ Team(const Team&)   <- implement
+ ~Team()
+ operator=(const Team&): Team&
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Exception
+ message: string
+ Exception(string)
+ what(): string
```

The other `Employee` members are in `Employee.h`.

Not on the published partial UML: `Team(const Team&)` (the target), `Team::operator=`, `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

How the given `Team` works:

- `members` is an array of `size` pointers. Each slot holds an `Employee*` or `NULL`. `NULL` means the slot is empty.
- The team owns its employees. `~Team()` deletes every employee and then the array.
- `Team::operator=` is given. It checks for self-assignment, deletes every employee in `members` and the old array, and then makes a deep copy of the other team (a new array, a new copy of every

employee, `NULL` slots stay `NULL`).

What the copy constructor must do

The copy constructor runs when a new team is made from an existing team, for example `Team b(a);`, `Team b = a;`, or when a `Team` is passed to a function by value. It makes a deep copy: the copy gets its own new objects; it does not share them with the original.

1. The new team gets the same `size` as the other team and a **new** array of that size.
2. For every slot of the other team: if the slot is `NULL`, the new slot is `NULL`. Otherwise, the new slot gets a **new copy** of that employee (use the `Employee` copy constructor). Every employee stays in the same slot number.
3. The two teams must not share any `Employee` object.

You may write the deep copy yourself, or you may reuse the given `operator=`.

Rules

- Implement only `Team::Team(const Team&)` in `A_3.cpp`. The declaration is already in `Team.h`.
- `getSize()` and `getMemberCount()` are not `const` functions, so use `other.size` and `other.members` directly. This is allowed inside the class's own constructor.
- Your code must compile with `-std=c++98 -Wall -Werror`. Memory is marked: `valgrind` must show no leaks and no invalid reads or frees.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `A_3` folder.
2. Run `make A_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `A_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Hints for this practice question are in `HINTS.md` in this folder.

B_1 Recursion

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B1-01. Recursion: `Employee::extractIntFromString` (100)

For this question, implement the static function `extractIntFromString` of the `Employee` class **using recursion**, in the file `B_1.cpp`. All the other functions of `Employee` and `Exception` are given in the other files.

Note: in this question, `extractIntFromString` returns `-1` for invalid input. It does not throw. The given `setIDNum` calls it and expects `-1` when a part of the ID number is not valid.

UML (only the classes this question needs)

```
Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}   <- implement
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Exception
+ message: string
+ Exception(string)
+ what(): string
```

Not on the published partial UML: the `int` return type of `extractIntFromString` (the UML shows no return type), `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

What `extractIntFromString(str, start, end)` must do

It reads the characters of `str` from index `start` up to, **but not including**, index `end`, and returns them as one whole number.

- Example: for `str = "9702140123081"`, `extractIntFromString(str, 0, 2)` returns `97`, and `extractIntFromString(str, 6, 10)` reads `"0123"` and returns `123`.
- Leading zeros are allowed: `"007"` gives `7`, and `"000"` gives `0`.
- Return `-1` if the range is invalid. The range is invalid when `start < 0`, or `start >= end` (an empty range), or `end` is more than the length of `str`.
- Return `-1` if **any** character in the range is not a digit (`'0'` to `'9'`). Characters outside the range are not checked.

- `0` is a real result (for example `"00"`), so it cannot be used to mean "invalid". That is why invalid input gives `-1` in this question.
- The range is never longer than 9 characters, so the result always fits in an `int`.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself; it may not call any other function you write.
- Implement only `Employee::extractIntFromString` in `B_1.cpp`. The declaration is already in `Employee.h`. Do not write `static` again in `B_1.cpp`.
- `str.length()` is unsigned. Cast it, `(int)str.length()`, before you compare it with an `int`, or `-Wall -Werror` stops the build.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Hints for this practice question are in `HINTS.md` in this folder.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B1-02. Recursion: `Employee::isAllDigits` (100)

For this question, implement the static function `isAllDigits` of the `Employee` class **using recursion**, in the file `B_1.cpp`. All the other functions of `Employee` and `Exception` are given in the other files. The given `setIDNum` uses `isAllDigits` to check an ID number.

UML (only the classes this question needs)

```
Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ isAllDigits(string, int): bool   {static}   <- implement
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Exception
+ message: string
+ Exception(string)
+ what(): string
```

Not on the published partial UML: `Employee::isAllDigits` (the target), the `int` return type of `extractIntFromString` (the UML shows no return type), `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

What `isAllDigits(str, index)` must do

It checks the characters of `str` from position `index` to the end of the string.

- Return `true` if every character from `index` to the end is a digit (`'0'` to `'9'`). Characters before `index` are not checked.
- Return `false` as soon as one of those characters is not a digit.
- When `index` is equal to the length of `str`, there is nothing left to check, so return `true`. (So `isAllDigits("", 0)` is `true`.)
- Invalid input: if `index` is less than `0` or more than the length of `str`, return `false`.

Examples: `isAllDigits("9702140123081", 0)` is `true`. `isAllDigits("abc123", 3)` is `true`. `isAllDigits("abc123", 0)` is `false`.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself; it may not call any other function you write.
- Implement only `Employee::isAllDigits` in `B_1.cpp`. The declaration is already in `Employee.h`. Do not write `static` again in `B_1.cpp`.
- `str.length()` is unsigned. Cast it, `(int)str.length()`, before you compare it with an `int`, or `-Wall -Werror` stops the build.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B1-03. Recursion: `Team::countMembers` (100)

For this question, implement the function `countMembers` of the `Team` class **using recursion**, in the file `B_1.cpp`. All the other functions of `Team`, `Employee` and `Exception` are given in the other files.

UML (only the classes this question needs)

```
Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ countMembers(int): int   <- implement
+ operator<<(ostream&, const Team&): ostream&   {friend}

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Exception
+ message: string
+ Exception(string)
+ what(): string
```

The other `Employee` members are in `Employee.h`.

Not on the published partial UML: `Team::countMembers` (the target), `Team::addMember`, `Team::removeMember`, `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

How the given `Team` works:

- `members` is an array of `size` pointers. Each slot holds an `Employee*` or `NULL`. `NULL` means the slot is empty.
- `addMember` puts an employee in the first empty slot. `removeMember(i)` deletes the employee in slot `i` and sets the slot to `NULL`, so empty slots can be anywhere in the array.

What `countMembers(index)` must do

It counts the slots that hold an employee (the slots that are not `NULL`), from slot `index` to the last slot (`size - 1`).

- `countMembers(0)` counts the whole team.

- When `index` is equal to `size`, there are no slots left, so return `0`.
- Invalid input: if `index` is less than `0` or more than `size`, return `0`.

Example: for a team of size 5 with slots `Thandi, empty, Anna, Ravi, empty`, `countMembers(0)` returns `3`, `countMembers(2)` returns `2` and `countMembers(4)` returns `0`.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself; it may not call any other function you write.
- Implement only `Team::countMembers` in `B_1.cpp`. The declaration is already in `Team.h`.
- Use the members `members` and `size` directly.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B1-04. Recursion: `Department::totalEmployees` (100)

For this question, implement the function `totalEmployees` of the `Department` class **using recursion**, in the file `B_1.cpp`. All the other functions of `Department`, `Team`, `Employee` and `Exception` are given in the other files.

UML (only the classes this question needs)

```
Department
- teams: Team**
- size: int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ totalEmployees(int): int   <- implement
+ operator<<(ostream&, Department&): ostream&   {friend}

Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ getMemberCount(): int
+ operator<<(ostream&, const Team&): ostream&   {friend}
```

`Employee` and `Exception` are the same as in the other questions. See `Employee.h` and `Exception.h`.

Not on the published partial UML: `Department::totalEmployees` (the target), `Department::getSize`, `Department::getName`, the `{virtual}` markers (the UML marks nothing virtual), `Team::addMember`, `Team::removeMember`, `Team::getMemberCount`, `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

How the given classes work:

- `teams` is an array of `size` pointers. Each slot holds a `Team*` or `NULL`. `NULL` means the slot is empty.
- `Team::getMemberCount()` is given. It returns the number of employees in that team (the member slots that are not `NULL`).

What `totalEmployees(index)` must do

It returns the total number of employees in the teams from slot `index` to the last slot (`size - 1`).

- An empty slot (`NULL`) adds `0`. A team with no employees also adds `0`.
- `totalEmployees(0)` counts every employee in the department.

- When `index` is equal to `size`, there are no slots left, so return `0`.
- Invalid input: if `index` is less than `0` or more than `size`, return `0`.

Example: for slots `team with 2 employees, empty, team with 1 employee, empty`, `totalEmployees(0)` returns `3`, `totalEmployees(2)` returns `1` and `totalEmployees(3)` returns `0`.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself; it may not call any other function you write. Calling the given `Team::getMemberCount()` is allowed.
- Implement only `Department::totalEmployees` in `B_1.cpp`. The declaration is already in `Department.h`.
- Use the members `teams` and `size` directly.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B1-05. Recursion: `List<T>::countNodes` (100)

For this question, implement the function `countNodes` of the class template `List<T>` **using recursion**, in the file `B_1.cpp`. All the other functions of `List<T>` and `Node<T>` are given in the other files.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ countNodes(Node<T>*): int   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `List<T>::countNodes` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

How the given classes work:

- `head()` returns the first node of the list, or `NULL` for an empty list.
- `next()` returns the next node, or `NULL` after the last node.
- `prepend(value)` adds a new node at the front of the list.
- `operator<<` prints `Empty List` for an empty list, or the values joined by `->`, for example `1->2->3`.

What `countNodes(curr)` must do

It returns the number of nodes from `curr` to the end of the list, counting `curr` itself.

- `list.countNodes(list.head())` counts the whole list.
- `list.countNodes(list.head()->next())` counts every node except the first one.
- If `curr` is `NULL`, there are no nodes, so return `0`. (This also covers an empty list.)

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_1.cpp` at the bottom (the COS 110 rule for templates).
- `B_1.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_1_CPP`, `#define B_1_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_1_CPP` and

`#endif`. Code after `#endif` gives a redefinition error.
- `B_1.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Your function is a template member. It starts like this:

```
template <class T>
int List<T>::countNodes(Node<T>* curr) {
```

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself; it may not call any other function you write.
- Do not use `size` or `getSize()`. They count the whole list, not the nodes from `curr`.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B1-06. Recursion: `List<T>::printReverse` (100)

For this question, implement the function `printReverse` of the class template `List<T>` **using recursion**, in the file `B_1.cpp`. All the other functions of `List<T>` and `Node<T>` are given in the other files.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ printReverse(Node<T>*): void   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `List<T>::printReverse` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

How the given classes work:

- `head()` returns the first node of the list, or `NULL` for an empty list.
- `next()` returns the next node, or `NULL` after the last node.
- `getData()` returns the value stored in the node.
- `operator<<` prints `Empty List` for an empty list, or the values joined by `->`, for example `1->2->3`.

What `printReverse(curr)` must do

It prints the values from the **last** node of the list back to `curr`.

- Print each value with `cout`, followed by `endl`. So each value is on its own line, and nothing else is printed.
- Example: for the list `1->2->3`, `printReverse(head())` prints `3`, `2` and `1`, each on its own line. `printReverse(head()->next())` prints `3` and `2`.
- If `curr` is `NULL`, print nothing. (This also covers an empty list.)
- Do not change the list.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_1.cpp` at the bottom (the COS 110 rule for templates).
- `B_1.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_1_CPP`, `#define B_1_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_1_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `B_1.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Your function is a template member. It starts like this:

```
template <class T>
void List<T>::printReverse(Node<T>* curr) {
```

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself; it may not call any other function you write.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B1-07. Recursion: `List<T>::contains` (100)

For this question, implement the function `contains` of the class template `List<T>` **using recursion**, in the file `B_1.cpp`. All the other functions of `List<T>` and `Node<T>` are given in the other files.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ contains(Node<T>*, T): bool   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `List<T>::contains` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

How the given classes work:

- `head()` returns the first node of the list, or `NULL` for an empty list.
- `next()` returns the next node, or `NULL` after the last node.
- `getData()` returns the value stored in the node.
- `operator<<` prints `Empty List` for an empty list, or the values joined by `->`, for example `1->2->3`.

What `contains(curr, value)` must do

It searches the nodes from `curr` to the end of the list.

- Return `true` if `curr` or any node after `curr` holds a value equal to `value`. Compare with `==`.
- Return `false` if no node from `curr` onwards holds `value`. Nodes before `curr` are not searched.
- If `curr` is `NULL`, return `false`. (This also covers an empty list.)
- Strings are compared exactly, so `"hr"` is not equal to `"HR"`.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_1.cpp` at the bottom (the COS 110 rule for templates).
- `B_1.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_1_CPP`, `#define B_1_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_1_CPP` and

`#endif`. Code after `#endif` gives a redefinition error.
- `B_1.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Your function is a template member. It starts like this:

```
template <class T>
bool List<T>::contains(Node<T>* curr, T value) {
```

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself; it may not call any other function you write.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B1-08. Recursion: `Company::totalCost` (100)

For this question, implement the function `totalCost` of the `Company` class **using recursion**, in the file `B_1.cpp`. All the other functions are given in the other files.

UML (only the classes this question needs)

```
Company
- departments: List<Department*>
+ Company()
+ ~Company()
+ addDepartment(Department*): void
+ getDepartments(): List<Department*>&
+ costToCompany(): float
+ totalCost(Node<Department*>*): float   <- implement
+ operator<<(ostream&, Company&): ostream&   {friend}

List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&

Department
- teams: Team**
- size: int
# countEmployees(): int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}

Sales : Department
+ Sales(int)
+ getName(): string
# costToCompany(): float

Marketing : Department
+ Marketing(int)
+ getName(): string
```

```
# costToCompany(): float

HR : Department
+ HR(int)
+ getName(): string
# costToCompany(): float

Development : Department
+ Development(int)
+ getName(): string
# costToCompany(): float
```

`Team`, `Employee` and `Exception` are the same as in the other questions. See `Team.h`, `Employee.h` and `Exception.h`.

Not on the published partial UML: `Company::totalCost` (the target), `~Company()` (given: it deletes every department, because the list frees only its nodes), `Company::getDepartments`, `Company::costToCompany`, `Department::countEmployees`, `Department::getSize`, `getName()` in `Department` and its subclasses, the `{virtual}` markers (the UML marks nothing virtual), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

How the given classes work:

- `addDepartment` adds a department at the **front** of the list (it uses `prepend`).
- For `List<Department*>`, each node stores a `Department*`, so `curr->getData()` gives a `Department*`.
- A node can store a `NULL` department. In that case `curr->getData()` is `NULL`.
- `costToCompany()` is `virtual`. Through a `Department*` it calls the cost of the real department type: the number of employees times R15000 (Sales), R18000 (Marketing), R12000 (HR) or R25000 (Development).
- The given `Company::costToCompany()` returns `totalCost(departments.head())`.

What `totalCost(curr)` must do

It returns the total cost of the department in node `curr` and of every department in the nodes after `curr`.

- Add `costToCompany()` of each department, from `curr` to the end of the list.
- If `curr->getData()` is `NULL` (no department), add `0` for that node.
- If `curr` is `NULL`, there are no nodes, so return `0`. (This also covers an empty company.)

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself; it may not call any other function you write. Calling the given `costToCompany()` of a department is allowed.
- Implement only `Company::totalCost` in `B_1.cpp`. The declaration is already in `Company.h`.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B1-09. Recursion: `Employee::checksum` (100)

For this question, implement the static function `checksum` of the `Employee` class **using recursion**, in the file `B_1.cpp`. All the other functions of `Employee` and `Exception` are given in the other files.

Note: a South African ID number ends with a check digit. The given `setIDNum` uses `checksum` to test that check digit. Only this question checks the check digit. In the other questions of this bank, an ID number such as `9702140123081` is accepted as a normal ID number.

UML (only the classes this question needs)

```
Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ checksum(string, int): int   {static}   <- implement
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Exception
+ message: string
+ Exception(string)
+ what(): string
```

Not on the published partial UML: `Employee::checksum` (the target), the `int` return type of `extractIntFromString` (the UML shows no return type), `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

The Luhn sum

The check uses the Luhn rule. Look at every digit of `id`, from left to right:

- Count the characters **after** this digit (to the end of `id`). That number is `id.length() - 1 - index`.
- If that number is **odd**, double the digit. If the doubled value is more than 9, subtract 9 from it.
- If that number is **even**, use the digit as it is. (So the last digit is never doubled.)

The Luhn sum is the total of these values. An ID number is valid when its Luhn sum is a multiple of 10.

Worked example for `"7918"`:

Index	Digit	Characters after it	Odd or even	Value
0	7	3	odd: 7 x 2 = 14, more than 9, so 14 - 9	5

Index	Digit	Characters after it	Odd or even	Value
1	9	2	even	9
2	1	1	odd: 1 x 2	2
3	8	0	even	8

The Luhn sum is `5 + 9 + 2 + 8 = 24`. It is not a multiple of 10, so `"7918"` is not valid. `checksum("7918", 2)` is `2 + 8 = 10`: the `1` is still doubled, because the characters after it are counted to the end of the whole string.

What `checksum(id, index)` must do

It returns the Luhn sum of the digits from position `index` to the end of `id`. The doubling rule always counts the characters after a digit up to the end of the **whole** string `id`.

- `checksum(id, 0)` is the Luhn sum of the whole ID number.
- When `index` is equal to the length of `id`, there are no digits left, so return `0`.
- Invalid input: if `index` is less than `0` or more than the length of `id`, return `0`.
- You may assume that `id` holds only digits. (`setIDNum` checks this before it calls `checksum`.)

Examples: `checksum("79927398713", 0)` is `70`. `checksum("9702140123086", 0)` is `50`, so that ID number is valid. `checksum("9702140123081", 0)` is `45`, so that one is not.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself; it may not call any other function you write.
- Implement only `Employee::checksum` in `B_1.cpp`. The declaration is already in `Employee.h`. Do not write `static` again in `B_1.cpp`.
- `id.length()` is unsigned. Cast it, `(int)id.length()`, before you use it with an `int`, or `-Wall -Werror` stops the build.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B1-10. Recursion: `Team::oldest` (100)

For this question, implement the function `oldest` of the `Team` class **using recursion**, in the file `B_1.cpp`. All the other functions of `Team`, `Employee` and `Exception` are given in the other files.

UML (only the classes this question needs)

```
Team
- members: Employee**
- size: int
+ Team(int)
+ ~Team()
+ addMember(Employee*): void
+ removeMember(int): void
+ operator[](int): const Employee&
+ getSize(): int
+ oldest(int): int   <- implement
+ operator<<(ostream&, const Team&): ostream&   {friend}

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getAge(): int
+ operator<<(ostream&, const Employee&): ostream&   {friend}

Exception
+ message: string
+ Exception(string)
+ what(): string
```

The other `Employee` members are in `Employee.h`.

Not on the published partial UML: `Team::oldest` (the target), `Team::addMember`, `Team::removeMember`, `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

How the given `Team` works:

- `members` is an array of `size` pointers. Each slot holds an `Employee*` or `NULL`. `NULL` means the slot is empty.
- `addMember` puts an employee in the first empty slot. `removeMember(i)` deletes the employee in slot `i` and sets the slot to `NULL`, so empty slots can be anywhere in the array.

What `oldest(index)` must do

It returns the age of the oldest employee in the slots from `index` to the last slot (`size - 1`).

- Skip empty (`NULL`) slots.

- If there is no employee in those slots, return `0`.
- When `index` is equal to `size`, there are no slots left, so return `0`.
- Invalid input: if `index` is less than `0` or more than `size`, return `0`.
- Ages are always more than `0`. If two employees have the same age, the answer is that age.

Example: for slots `Thandi (29), empty, Anna (41), Ravi (38), empty`, `oldest(0)` returns `41`, `oldest(3)` returns `38` and `oldest(4)` returns `0`.

Rules

- You must use recursion. **No loops (`for`, `while`, `do`) and no helper functions. Using either scores 0.** The function must call itself; it may not call any other function you write. Calling the given `Employee::getAge()` is allowed.
- Implement only `Team::oldest` in `B_1.cpp`. The declaration is already in `Team.h`.
- Use the members `members` and `size` directly.
- Your code must compile with `-std=c++98 -Wall -Werror` and must not leak memory.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_1` folder.
2. Run `make B_1`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_1.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

B_2 Linked lists

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B2-01. Linked lists: `List<T>::append` (100)

For this question, write a function that adds a value at the end of a singly linked list. Write the `append` function of the class template `List<T>` in the file `B_2.cpp`.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ append(T): void   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `append` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores a **pointer** to its value (`data: T*`). The node owns that value: `~Node()` deletes `data`. `~Node()` does **not** delete the next node.
- To make a node for a value, write `new Node<T>(new T(value))`. The given `prepend` does it this way.
- `next()` returns a reference to the link, so you can change a link with `curr->next() = ...;`.
- `getData()` returns the value itself (a `const T&`), not the pointer.
- `size` must always be equal to the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty List` for an empty list. Otherwise it prints the values joined by `->`, for example `1->2->3`. It does not print a newline.
- Inside `List` you can use `headNode` directly.

Behaviour of `append(T value)`

- Make a new node that holds a copy of `value`.
- If the list is empty, the new node becomes the head.

- Otherwise, link the new node after the current last node. The order of the other nodes must not change.
- The new node is the last node, so its `next()` stays `NULL`.
- Add 1 to `size`.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_2.cpp` at the bottom (the COS 110 rule for templates).
- `B_2.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_2_CPP`, `#define B_2_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_2_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `B_2.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Write the function as a template member. It starts like this:

```
template <class T>
void List<T>::append(T value) {
```

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B2-02. Linked lists: `List<T>::insertAt` (100)

For this question, write a function that inserts a value at a given position in a singly linked list. Write the `insertAt` function of the class template `List<T>` in the file `B_2.cpp`.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ insertAt(T, int): void   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `insertAt` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores a **pointer** to its value (`data: T*`). The node owns that value: `~Node()` deletes `data`. `~Node()` does **not** delete the next node.
- To make a node for a value, write `new Node<T>(new T(value))`. The given `prepend` does it this way.
- `next()` returns a reference to the link, so you can change a link with `curr->next() = ...;`.
- `getData()` returns the value itself (a `const T&`), not the pointer.
- `size` must always be equal to the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty List` for an empty list. Otherwise it prints the values joined by `->`, for example `1->2->3`. It does not print a newline.
- Inside `List` you can use `headNode` directly.

Behaviour of `insertAt(T value, int pos)`

- Positions start at 0. The head node is at position 0.
- Make a new node that holds a copy of `value`. After the call, the new node must be at position `pos`. The nodes that were at `pos` and after it move one place to the right.
- If `pos` is 0 or less, the new node becomes the head.
- If `pos` is equal to or greater than the number of nodes, the new node is added at the end. This also covers an empty list.

- Add 1 to `size`.

Example: for `10->20->30`, `insertAt(15, 1)` gives `10->15->20->30`, and `insertAt(99, 7)` gives `10->20->30->99`.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_2.cpp` at the bottom (the COS 110 rule for templates).
- `B_2.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_2_CPP`, `#define B_2_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_2_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `B_2.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Write the function as a template member. It starts like this:

```
template <class T>
void List<T>::insertAt(T value, int pos) {
```

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B2-03. Linked lists: `List<T>::remove` by value (100)

For this question, write a function that removes a value from a singly linked list. Write the `remove` function of the class template `List<T>` in the file `B_2.cpp`.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ remove(T): bool   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `remove` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores a **pointer** to its value (`data: T*`). The node owns that value: `~Node()` deletes `data`. `~Node()` does **not** delete the next node.
- To make a node for a value, write `new Node<T>(new T(value))`. The given `prepend` does it this way.
- `next()` returns a reference to the link, so you can change a link with `curr->next() = ...;`.
- `getData()` returns the value itself (a `const T&`), not the pointer.
- `size` must always be equal to the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty List` for an empty list. Otherwise it prints the values joined by `->`, for example `1->2->3`. It does not print a newline.
- Inside `List` you can use `headNode` directly.

Behaviour of `remove(T value)`

- Find the **first** node (starting from the head) whose data is equal to `value` (use `==`).
- Unlink that node from the list and delete it. Deleting the node also deletes its data, because `~Node()` deletes `data`. Do not delete the data yourself.
- Subtract 1 from `size` and return `true`.
- If no node holds `value`, or the list is empty, change nothing and return `false`.

- Your function must work for every case: an empty list, a list with one node, the head, the last node and a node in the middle.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_2.cpp` at the bottom (the COS 110 rule for templates).
- `B_2.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_2_CPP`, `#define B_2_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_2_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `B_2.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Write the function as a template member. It starts like this:

```
template <class T>
bool List<T>::remove(T value) {
```

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B2-04. Linked lists: `List<T>::removeAt` (100)

For this question, write a function that removes the node at a given position from a singly linked list. Write the `removeAt` function of the class template `List<T>` in the file `B_2.cpp`.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ removeAt(int): bool   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `removeAt` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores a **pointer** to its value (`data: T*`). The node owns that value: `~Node()` deletes `data`. `~Node()` does **not** delete the next node.
- To make a node for a value, write `new Node<T>(new T(value))`. The given `prepend` does it this way.
- `next()` returns a reference to the link, so you can change a link with `curr->next() = ...;`.
- `getData()` returns the value itself (a `const T&`), not the pointer.
- `size` must always be equal to the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty List` for an empty list. Otherwise it prints the values joined by `->`, for example `1->2->3`. It does not print a newline.
- Inside `List` you can use `headNode` directly.

Behaviour of `removeAt(int pos)`

- Positions start at 0. The head node is at position 0. The last node is at position `size - 1`.
- If `pos` is less than 0, or `pos` is equal to or greater than the number of nodes, change nothing and return `false`. An empty list always returns `false`.
- Otherwise, unlink the node at position `pos` and delete it. Deleting the node also deletes its data (`~Node()` does that), so do not delete the data yourself.
- Subtract 1 from `size` and return `true`.

Example: for `10->20->30`, `removeAt(1)` returns `true` and the list becomes `10->30`. `removeAt(3)` returns `false`.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_2.cpp` at the bottom (the COS 110 rule for templates).
- `B_2.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_2_CPP`, `#define B_2_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_2_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `B_2.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Write the function as a template member. It starts like this:

```
template <class T>
bool List<T>::removeAt(int pos) {
```

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B2-05. Linked lists: `List<T>::reverse` (100)

For this question, write a function that reverses a singly linked list. Write the `reverse` function of the class template `List<T>` in the file `B_2.cpp`.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ reverse(): void   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `reverse` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores a **pointer** to its value (`data: T*`). The node owns that value: `~Node()` deletes `data`. `~Node()` does **not** delete the next node.
- To make a node for a value, write `new Node<T>(new T(value))`. The given `prepend` does it this way.
- `next()` returns a reference to the link, so you can change a link with `curr->next() = ...;`.
- `getData()` returns the value itself (a `const T&`), not the pointer.
- `size` must always be equal to the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty List` for an empty list. Otherwise it prints the values joined by `->`, for example `1->2->3`. It does not print a newline.
- Inside `List` you can use `headNode` directly.

Behaviour of `reverse()`

- After the call, the nodes are in the opposite order. The last node becomes the head, and the old head becomes the last node (its `next()` must be `NULL`).
- Change only the links (`next()` of each node and `headNode`). Do not create new nodes and do not delete nodes. The marking program checks that the same nodes are still used.
- `size` does not change.
- An empty list and a list with one node stay the same.

Example: `1->2->3->4` becomes `4->3->2->1`.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_2.cpp` at the bottom (the COS 110 rule for templates).
- `B_2.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_2_CPP`, `#define B_2_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_2_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `B_2.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Write the function as a template member. It starts like this:

```
template <class T>
void List<T>::reverse() {
```

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B2-06. Linked lists: `List<T>::sort` by relinking (100)

For this question, write a function that sorts a singly linked list. Write the `sort` function of the class template `List<T>` in the file `B_2.cpp`. This function sorts the current list in ascending order.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ sort(): void   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `sort` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores a **pointer** to its value (`data: T*`). The node owns that value: `~Node()` deletes `data`. `~Node()` does **not** delete the next node.
- To make a node for a value, write `new Node<T>(new T(value))`. The given `prepend` does it this way.
- `next()` returns a reference to the link, so you can change a link with `curr->next() = ...;`.
- `getData()` returns the value itself (a `const T&`), not the pointer.
- `size` must always be equal to the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty List` for an empty list. Otherwise it prints the values joined by `->`, for example `1->2->3`. It does not print a newline.
- Inside `List` you can use `headNode` directly.

Behaviour of `sort()`

- After the call, the values are in ascending order (smallest at the head). You may use the comparison operators `<`, `<=`, `>` and `>=` on `T`. For strings this gives alphabetical order.
- If two values are equal, either order is accepted.
- `getData()` returns a `const T&` and `Node` has no setter, so you **cannot** change or swap the data. You must move whole nodes by changing the links (`next()` of the nodes and `headNode`).

- Do not create new nodes and do not delete nodes. The marking program checks that the same nodes are still used.
- The last node's `next()` must be `NULL` after the sort.
- `size` does not change. An empty list and a list with one node stay the same.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_2.cpp` at the bottom (the COS 110 rule for templates).
- `B_2.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_2_CPP`, `#define B_2_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_2_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `B_2.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Write the function as a template member. It starts like this:

```
template <class T>
void List<T>::sort() {
```

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B2-07. Linked lists: `List<T>::indexOf` (100)

For this question, write a function that searches a singly linked list. Write the `indexOf` function of the class template `List<T>` in the file `B_2.cpp`.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ indexOf(T): int   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `indexOf` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores a **pointer** to its value (`data: T*`). The node owns that value: `~Node()` deletes `data`. `~Node()` does **not** delete the next node.
- To make a node for a value, write `new Node<T>(new T(value))`. The given `prepend` does it this way.
- `next()` returns a reference to the link, so you can change a link with `curr->next() = ...;`.
- `getData()` returns the value itself (a `const T&`), not the pointer.
- `size` must always be equal to the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty List` for an empty list. Otherwise it prints the values joined by `->`, for example `1->2->3`. It does not print a newline.
- Inside `List` you can use `headNode` directly.

Behaviour of `indexOf(T value)`

- Positions start at 0. The head node is at position 0.
- Return the position of the **first** node (starting from the head) whose data is equal to `value` (use `==`).
- If no node holds `value`, or the list is empty, return `-1`.
- Do not change the list.

Example: for `Sales->HR->Sales`, `indexOf("Sales")` returns `0` and `indexOf("IT")` returns `-1`.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_2.cpp` at the bottom (the COS 110 rule for templates).
- `B_2.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_2_CPP`, `#define B_2_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_2_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `B_2.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Write the function as a template member. It starts like this:

```
template <class T>
int List<T>::indexOf(T value) {
```

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B2-08. Linked lists: `List<T>::operator[]` (100)

For this question, write the subscript operator for a singly linked list. Write `operator[]` of the class template `List<T>` in the file `B_2.cpp`.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ operator[](int): const T&   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&

Exception
+ message: string
+ Exception(string)
+ what(): string
+ operator<<(ostream&, const Exception&): ostream&   {friend}
```

Not on the published partial UML: `operator[]` of `List<T>` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores a **pointer** to its value (`data: T*`). The node owns that value: `~Node()` deletes `data`. `~Node()` does **not** delete the next node.
- To make a node for a value, write `new Node<T>(new T(value))`. The given `prepend` does it this way.
- `next()` returns a reference to the link, so you can change a link with `curr->next() = ...;`.
- `getData()` returns the value itself (a `const T&`), not the pointer.
- `size` must always be equal to the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty List` for an empty list. Otherwise it prints the values joined by `->`, for example `1->2->3`. It does not print a newline.
- Inside `List` you can use `headNode` directly.

Behaviour of `operator[](int index)`

- Positions start at 0. The head node is at position 0. The last node is at position `size - 1`.
- If `index` is less than 0, or equal to or greater than the number of nodes, throw an `Exception` with the message `Index out of bounds`. Throw it by value: `throw Exception("Index out of bounds");`. An empty list always throws.
- Otherwise, return the data of the node at position `index`. Return a reference to the data stored in the node (use `getData()`), not a reference to a local copy.
- Do not change the list.

`List.h` already includes `Exception.h`, and the makefile compiles `Exception.cpp`.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_2.cpp` at the bottom (the COS 110 rule for templates).
- `B_2.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_2_CPP`, `#define B_2_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_2_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `B_2.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Write the function as a template member. It starts like this:

```
template <class T>
const T& List<T>::operator[](int index) {
```

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B2-09. Linked lists: `List<T>::clear` (100)

For this question, write a function that deletes all the nodes of a singly linked list. Write the `clear` function of the class template `List<T>` in the file `B_2.cpp`.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ clear(): void   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: `clear` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores a **pointer** to its value (`data: T*`). The node owns that value: `~Node()` deletes `data`. `~Node()` does **not** delete the next node.
- To make a node for a value, write `new Node<T>(new T(value))`. The given `prepend` does it this way.
- `next()` returns a reference to the link, so you can change a link with `curr->next() = ...;`.
- `getData()` returns the value itself (a `const T&`), not the pointer.
- `size` must always be equal to the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty List` for an empty list. Otherwise it prints the values joined by `->`, for example `1->2->3`. It does not print a newline.
- Inside `List` you can use `headNode` directly.
- The given destructor `~List()` calls `clear()`. So if `clear()` leaks memory, the whole program leaks memory.

Behaviour of `clear()`

- Delete every node of the list. Deleting a node also deletes its data (`~Node()` does that), so do not delete the data yourself.
- `~Node()` does not delete the next node, so you must visit every node.
- After the call, `headNode` is `NULL` and `size` is 0.

- The list must still work after `clear()`: values can be added again, and `clear()` can be called again (also on an empty list).

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_2.cpp` at the bottom (the COS 110 rule for templates).
- `B_2.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_2_CPP`, `#define B_2_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_2_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `B_2.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Write the function as a template member. It starts like this:

```
template <class T>
void List<T>::clear() {
```

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B2-10. Linked lists: `Company::removeDepartment` (ownership when removing from a list) (100)

For this question, write a function that removes a department from a company. A company keeps its departments in a linked list (`List<Department*>`). Write the `removeDepartment` function of the `Company` class in the file `B_2.cpp`.

UML (only the classes this question needs)

```
Company
- departments: List<Department*>
+ Company()
+ ~Company()
+ addDepartment(Department*): void
+ costToCompany(): float
+ removeDepartment(string): bool   <- implement
+ operator<<(ostream&, Company&): ostream&   {friend}

List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ remove(T): bool
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&

Department
- teams: Team**
- size: int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}
```

`Sales`, `HR`, `Marketing` and `Development` inherit from `Department`. `getName()` returns `"Sales"`, `"HR"`, `"Marketing"` or `"Development"`.

Not on the published partial UML: `~Company()`, `Company::costToCompany`, `removeDepartment` (the target), `List::remove`, `Department::getSize`, `Department::getName` and `getName()` in the

subclasses, the `{virtual}` markers (the UML marks nothing virtual), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- `addDepartment` uses `prepend`, so the list order is the **reverse** of the order in which departments were added.
- **Ownership:** in this question the company owns its departments. `~Company()` deletes every Department that is still in the list. A Department deletes its teams, and a Team deletes its employees. (Other questions in this bank may give this job to a different class.)
- `List<Department*>` does **not** delete the Departments. Each node holds a pointer to a `Department*` (`data` is a `Department**`). `~Node()` deletes only that pointer, not the Department.
- `List<T>::remove(T value)` is given. It unlinks and deletes the first node whose data `== value`, updates `size`, and returns `true` (or `false` if there is no such node). For `List<Department*>` it compares the pointers.
- `departments.head()` returns the first node (or `NULL`). For a node `curr`, `curr->getData()` returns the `Department*` and `curr->next()` returns the next node.

Behaviour of `removeDepartment(string name)`

- Find the **first** department in the list (starting from the head) whose `getName()` is equal to `name`. Names are case sensitive.
- Remove its node from the list **and** delete the Department object. After the call there must be no leak and no pointer to memory that was already deleted.
- Return `true` if a department was removed. If no department has that name, or the company has no departments, change nothing and return `false`.
- Do not use a node after it has been deleted.

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.
- `Company` is not a template, so `B_2.cpp` is compiled on its own (it is on the g++ line). `List.h` does not include `B_2.cpp` in this question.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B2-11. Linked lists: `List<T>` copy constructor (100)

For this question, write the copy constructor of a singly linked list. Write the copy constructor of the class template `List<T>` in the file `B_2.cpp`.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ List(const List<T>&)   <- implement
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&
```

Not on the published partial UML: the copy constructor `List(const List<T>&)` (the target), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- `Node<T>` stores a **pointer** to its value (`data: T*`). The node owns that value: `~Node()` deletes `data`. `~Node()` does **not** delete the next node.
- To make a node for a value, write `new Node<T>(new T(value))`. The given `prepend` does it this way.
- `next()` returns a reference to the link, so you can change a link with `curr->next() = ...;`.
- `getData()` returns the value itself (a `const T&`), not the pointer.
- `size` must always be equal to the number of nodes. `getSize()` returns it.
- `operator<<` prints `Empty List` for an empty list. Otherwise it prints the values joined by `->`, for example `1->2->3`. It does not print a newline.
- Inside `List` you can use `headNode` directly.

Behaviour of `List(const List<T>& other)`

- Build a **deep copy** of `other` (the copy gets its own new nodes and its own new data; it shares nothing with `other`): make a new node for every node of `other`, and give each new node a new copy of the data (`new T(...)`).
- The copy must have the same values in the same order, and the same `size`.

- The copy must not share any node or any data with `other`. Changing or deleting one list afterwards must not change the other list.
- Copying an empty list gives an empty list.
- Remember that this is a constructor: `headNode` and `size` have no value yet when the body starts.

File layout (templates)

- Template code must be visible wherever the template is used. So `List.h` includes `List.cpp` and then `B_2.cpp` at the bottom (the COS 110 rule for templates).
- `B_2.cpp` is also on the g++ line in the makefile. Its include guard (`#ifndef B_2_CPP`, `#define B_2_CPP`, `#endif`) makes both ways work. Keep your code between `#define B_2_CPP` and `#endif`. Code after `#endif` gives a redefinition error.
- `B_2.cpp` already has `#include "List.h"`. Do not add `#include "List.cpp"`.
- In the real exam, follow the given headers and makefile of the exam.

Write the function as a template member. It starts like this:

```
template <class T>
List<T>::List(const List<T>& other)
```

Rules

- C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`.
- Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build.
- Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_2` folder.
2. Run `make B_2`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_2.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

B_3 Template specialization

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B3-01. Template specialization: `printItem<Department*>` (100)

For this question, write a template specialization (a version of a template written for one exact type). The function template `printItem` prints any value with its stream insertion operator. For a `Department*`, this would print an address. Write the specialization of `printItem` for `Department*` in the file `B_3.cpp`, so that it prints the Department itself.

UML (only the classes this question needs)

```
Templates.h (function templates)
+ printItem<T>(const T&): void
+ printItem<Department*>(Department* const&): void   <- implement

Department
- teams: Team**
- size: int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}
```

Not on the published partial UML: `printItem` and its specialization (the target; function templates are not on the UML), `Department::getSize`, `Department::getName`, the `{virtual}` markers (the UML marks nothing virtual), `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

What is given

- The general template in `Templates.h`: `cpp template <class T> void printItem(const T& item) { cout << item << endl; }`
- With `T = Department*`, the parameter type `const T&` becomes `Department* const&` (a reference to a constant pointer). It is **not** `const Department*&`. The declaration in `Templates.h` already uses the right type: `cpp template <> void printItem<Department*>(Department* const& item);`
- `Sales`, `HR` and `Development` inherit from `Department`. `costToCompany()` is virtual. It returns the number of employees in the department times a monthly amount: Sales R15000, HR R12000, Development R25000. The base `Department` returns 0.
- `operator<<` for a Department prints `<name> (cost R<cost>)` and a newline, and then every team (`Team:` and one line per employee). It already ends every line, so do not add an extra `endl` after it.
- A Department owns its teams, and a Team owns its employees.

Behaviour of `printItem<Department*>(Department* const& item)`

- If `item` is `NULL`, print `No department` and a newline.
- Otherwise, print the Department that `item` points to, using the Department stream insertion operator. Do not print the address. Do not add an extra newline, because the Department operator already ends its lines.

File layout

- `Templates.h` holds the general template (fully written) and, after it, a **declaration** of the specialization (only the header line, ending in `;`).
- You write the **definition** of the specialization in `B_3.cpp`. A full specialization is an ordinary function, not a template, so it must be defined in exactly one `.cpp` file. `B_3.cpp` is compiled on its own (it is on the g++ line).
- Because every file that includes `Templates.h` sees the declaration, `main.cpp` calls your specialization and not the general template.

Rules

- Write the specialization the way it is shown in the lectures (L19): the line `template <>`, then the function with the concrete type in angle brackets after the function name. It must match the declaration in `Templates.h` exactly (return type, name, `<type>` and parameter types). Your definition starts with the same two lines as the declaration, without the `;`:

`cpp template <> void printItem<Department*>(Department* const& item) {` - Write only the definition of the specialization in `B_3.cpp`. Do not write the general template again, and do not change any header. - C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`. - Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build. - Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B3-02. Template specialization: `largest<Employee>` (100)

For this question, write a template specialization (a version of a template written for one exact type). The function template `largest` returns the larger of two values with the `>` operator. The `Employee` class has no `>` operator, so the general template cannot be used for employees. Write the specialization of `largest` for `Employee` in the file `B_3.cpp`.

UML (only the classes this question needs)

```
Templates.h (function templates)
+ largest<T>(T, T): T
+ largest<Employee>(Employee, Employee): Employee   <- implement

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
+ operator<<(ostream&, const Employee&): ostream&   {friend}
```

Not on the published partial UML: `largest` and its specialization (the target; function templates are not on the UML), the `int` return type of `extractIntFromString` (the UML shows no return type), `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

What is given

- The general template in `Templates.h`: `cpp template <class T> T largest(T a, T b) { if (a > b) { return a; } return b; }`
- The declaration of the specialization, also in `Templates.h`: `cpp template <> Employee largest<Employee>(Employee a, Employee b);`
- `Employee` has a copy constructor, so employees can be passed and returned by value.
- `operator<<` for an Employee prints `<firstName> <surName>, <age>, <gender>, ID: <idNum>` (or `ID: none` when no ID number is set).

Behaviour of `largest<Employee>(Employee a, Employee b)`

- Compare the two employees by age (`getAge()`).
- Return the older employee.
- If both have the same age, return `b`. This is the same rule as the general template, which returns `b` when the values are equal.

File layout

- `Templates.h` holds the general template (fully written) and, after it, a **declaration** of the specialization (only the header line, ending in `;`).
- You write the **definition** of the specialization in `B_3.cpp`. A full specialization is an ordinary function, not a template, so it must be defined in exactly one `.cpp` file. `B_3.cpp` is compiled on its own (it is on the g++ line).
- Because every file that includes `Templates.h` sees the declaration, `main.cpp` calls your specialization and not the general template.

Rules

- Write the specialization the way it is shown in the lectures (L19): the line `template <>`, then the function with the concrete type in angle brackets after the function name. It must match the declaration in `Templates.h` exactly (return type, name, `<type>` and parameter types). Your definition starts with the same two lines as the declaration, without the `;`:

`cpp template <> Employee largest<Employee>(Employee a, Employee b) {` - Write only the definition of the specialization in `B_3.cpp`. Do not write the general template again, and do not change any header. - C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`. - Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build. - Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B3-03. Template specialization: `isEqual<Department*>` (100)

For this question, write a template specialization (a version of a template written for one exact type). The function template `isEqual` compares two values with `==`. For two `Department*` values this only compares the addresses. Write the specialization of `isEqual` for `Department*` in the file `B_3.cpp`, so that two departments are compared by their cost to the company.

UML (only the classes this question needs)

```
Templates.h (function templates)
+ isEqual<T>(T, T): bool
+ isEqual<Department*>(Department*, Department*): bool   <- implement

Department
- teams: Team**
- size: int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}
```

Not on the published partial UML: `isEqual` and its specialization (the target; function templates are not on the UML), `Department::getSize`, `Department::getName`, the `{virtual}` markers (the UML marks nothing virtual), `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

What is given

- The general template in `Templates.h`: `cpp template <class T> bool isEqual(T a, T b) { return a == b; }`
- The declaration of the specialization, also in `Templates.h`: `cpp template <> bool isEqual<Department*>(Department* a, Department* b);`
- `Sales`, `HR`, `Marketing` and `Development` inherit from `Department`. `costToCompany()` is virtual. It returns the number of employees in the department times a monthly amount: Sales R15000, HR R12000, Marketing R18000, Development R25000. The base `Department` returns 0.
- `operator<<` for a Department prints `<name> (cost R<cost>)` and a newline, and then every team (`Team:` and one line per employee). It already ends every line, so do not add an extra `endl` after it.
- A Department owns its teams, and a Team owns its employees.

Behaviour of `isEqual<Department*>(Department* a, Department* b)`

- If both pointers are `NULL`, return `true`.
- If only one of them is `NULL`, return `false`.
- Otherwise, return `true` if `a->costToCompany()` is equal to `b->costToCompany()`, and `false` if not. The type of department does not matter, only the cost.
- Do not delete or change the departments.

File layout

- `Templates.h` holds the general template (fully written) and, after it, a **declaration** of the specialization (only the header line, ending in `;`).
- You write the **definition** of the specialization in `B_3.cpp`. A full specialization is an ordinary function, not a template, so it must be defined in exactly one `.cpp` file. `B_3.cpp` is compiled on its own (it is on the g++ line).
- Because every file that includes `Templates.h` sees the declaration, `main.cpp` calls your specialization and not the general template.

Rules

- Write the specialization the way it is shown in the lectures (L19): the line `template <>`, then the function with the concrete type in angle brackets after the function name. It must match the declaration in `Templates.h` exactly (return type, name, `<type>` and parameter types). Your definition starts with the same two lines as the declaration, without the `;`:

`cpp template <> bool isEqual<Department*>(Department* a, Department* b) {` - Write only the definition of the specialization in `B_3.cpp`. Do not write the general template again, and do not change any header. - C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`. - Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build. - Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B3-04. Template specialization: `List<Department*>::printAll` (100)

For this question, write a specialization of one member function of a class template (a version of that member written for one exact type). `List<T>::printAll()` prints every value in the list. A company keeps its departments in a `List<Department*>`, and for that type the general version would print addresses. Write the specialization `List<Department*>::printAll()` in the file `B_3.cpp`.

UML (only the classes this question needs)

```
List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ printAll(): void
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

List<Department*>   (specialization of one member)
+ printAll(): void   <- implement

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&

Department
- teams: Team**
- size: int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}
```

Not on the published partial UML: `List::printAll` (the general version) and its specialization for `Department*` (the target), `Department::getSize`, `Department::getName`, the `{virtual}` markers (the UML marks nothing virtual), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- The general `printAll()` in `List.cpp`: `cpp template <class T> void List<T>::printAll() { if (headNode == NULL) { cout << "Empty List" << endl; return; } Node<T>* curr =`

`headNode; int index = 0; while (curr != NULL) { cout << index << ": " << curr->getData() << endl; curr = curr->next(); index++; } }`

- The declaration of the specialization at the bottom of `List.h`: `cpp template <> void List<Department*>::printAll();`
- `Node<T>` stores a pointer to its value (`data: T*`). For `List<Department*>`, `getData()` returns the `Department*`. `next()` returns the next node.
- The list does **not** own the Departments in this question. `main` deletes them.
- `Sales`, `HR` and `Development` inherit from `Department`. `costToCompany()` is virtual. It returns the number of employees in the department times a monthly amount: Sales R15000, HR R12000, Development R25000. The base `Department` returns 0.
- `operator<<` for a Department prints `<name> (cost R<cost>)` and a newline, and then every team (`Team:` and one line per employee). It already ends every line, so do not add an extra `endl` after it.
- A Department owns its teams, and a Team owns its employees.

Behaviour of `List<Department*>::printAll()`

- If the list is empty, print `Empty List` and a newline (the same as the general version).
- Otherwise, for each node from the head, with `index` counting from 0:
- if the `Department*` is `NULL`, print `<index>: No department` and a newline;
- otherwise, print `<index>:` and then the Department itself with the Department stream insertion operator. Do not add an extra newline, because the Department operator already ends its lines.
- Do not change the list and do not delete anything.

File layout

- `List.h` includes `List.cpp` at the bottom (the COS 110 rule for templates), so `List.cpp` is not on the g++ line.
- After that include, `List.h` **declares** the specialization of one member for `T = Department*` (only the header line, ending in `;`). `List.h` has `class Department;` at the top so that it can name the type.
- You write the **definition** in `B_3.cpp`. A full specialization is an ordinary function, not a template, so it must be defined in exactly one `.cpp` file. `B_3.cpp` is compiled on its own (it is on the g++ line).
- A member specialization starts with `template <>`. Then write `List<Department*>::printAll()`. Do not write `template <class T>` and do not write `<T>`. Inside this function `T` means `Department*`, so the nodes have the type `Node<Department*>*`. You can use `headNode` and `size` directly.

Rules

- Write the specialization like this: the line `template <>`, then `void List<Department*>::printAll()` and the body. The type goes after the class name `List`, not after `printAll`. It must match the declaration at the bottom of `List.h` exactly. Your definition starts like this:

`cpp template <> void List<Department*>::printAll() {` - Write only the definition of the specialization in `B_3.cpp`. Do not write the general template again, and do not change any header. - C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`. - Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build. - Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B3-05. Template specialization: `List<Department*>::clear` (100)

For this question, write a specialization of one member function of a class template (a version of that member written for one exact type). `List<T>::clear()` deletes every node of the list, and `~List()` calls `clear()`. In this question the company keeps its departments in a `List<Department*>`, and `Company` has no destructor. So the list must delete the Departments too. Write the specialization `List<Department*>::clear()` in the file `B_3.cpp`.

UML (only the classes this question needs)

```
Company
- departments: List<Department*>
+ Company()
+ addDepartment(Department*): void
+ costToCompany(): float
+ operator<<(ostream&, Company&): ostream&   {friend}

List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ clear(): void
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

List<Department*>   (specialization of one member)
+ clear(): void   <- implement

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&

Department
- teams: Team**
- size: int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}
```

Not on the published partial UML: `List::clear` (the general version) and its specialization for `Department*` (the target), `Company::costToCompany`, `Department::getSize`, `Department::getName`, the `{virtual}` markers (the UML marks nothing virtual), `operator<<` (not drawn; the scenario says every

class has one). `head()` is written `Node<T>* const& head() const`: a call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- The general `clear()` in `List.cpp` deletes every node, then sets `headNode` to `NULL` and `size` to 0. `~List()` calls `clear()`.
- The declaration of the specialization at the bottom of `List.h`: `cpp template <> void List<Department*>::clear();`
- `Node<T>` stores a pointer to its value (`data: T*`) and `~Node()` deletes `data`. For `List<Department*>`, `data` is a `Department**`. So deleting a node deletes only the node's copy of the pointer, **not** the Department. `getData()` returns the `Department*`. `~Node()` does not delete the next node.
- In this question `Company` has no destructor (the published UML shows none). When a Company is destroyed, its list is destroyed, and `~List()` calls `clear()`. Who deletes the departments is different in some other questions of this bank. Here it is `List<Department*>::clear()`.
- `Sales`, `HR`, `Marketing` and `Development` inherit from `Department`. `costToCompany()` is virtual. It returns the number of employees in the department times a monthly amount: Sales R15000, HR R12000, Marketing R18000, Development R25000. The base `Department` returns 0.
- `operator<<` for a Department prints `<name> (cost R<cost>)` and a newline, and then every team (`Team:` and one line per employee). It already ends every line, so do not add an extra `endl` after it.
- A Department owns its teams, and a Team owns its employees. `~Department()` is virtual, so deleting a `Sales` object through a `Department*` is safe.

Behaviour of `List<Department*>::clear()`

- For every node: delete the Department that the node holds, and delete the node. A `NULL` entry has no Department to delete.
- After the call, `headNode` is `NULL` and `size` is 0.
- The list must still work after `clear()`: departments can be added again, and `clear()` can be called again (also on an empty list).
- valgrind must report no leaks: every Department, Team and Employee must be freed exactly once.

File layout

- `List.h` includes `List.cpp` at the bottom (the COS 110 rule for templates), so `List.cpp` is not on the g++ line.
- After that include, `List.h` **declares** the specialization of one member for `T = Department*` (only the header line, ending in `;`). `List.h` has `class Department;` at the top so that it can name the type.
- You write the **definition** in `B_3.cpp`. A full specialization is an ordinary function, not a template, so it must be defined in exactly one `.cpp` file. `B_3.cpp` is compiled on its own (it is on the g++ line).
- A member specialization starts with `template <>`. Then write `List<Department*>::clear()`. Do not write `template <class T>` and do not write `<T>`. Inside this function `T` means `Department*`, so the nodes have the type `Node<Department*>*`. You can use `headNode` and `size` directly.

Rules

- Write the specialization like this: the line `template <>`, then `void List<Department*>::clear()` and the body. The type goes after the class name `List`, not after `clear`. It must match the declaration at the bottom of `List.h` exactly. Your definition starts like this:

`cpp template <> void List<Department*>::clear() {` - Write only the definition of the specialization in `B_3.cpp`. Do not write the general template again, and do not change any header. - C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`. - Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build. - Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B3-06. Template specialization: `totalCost<Department*>` (100)

For this question, write a template specialization (a version of a template written for one exact type). The function template `totalCost` returns the total monthly cost of the things in a list. For most types this is 0. Write the specialization of `totalCost` for `Department*` in the file `B_3.cpp`. `Company::costToCompany()` also uses it.

UML (only the classes this question needs)

```
Templates.h (function templates)
+ totalCost<T>(List<T>&): float
+ totalCost<Department*>(List<Department*>&): float   <- implement

Company
- departments: List<Department*>
+ Company()
+ ~Company()
+ addDepartment(Department*): void
+ costToCompany(): float
+ operator<<(ostream&, Company&): ostream&   {friend}

List<T>
- headNode: Node<T>*
- size: int
+ List()
+ ~List()
+ prepend(T): void
+ head(): Node<T>* const&   {const}
+ getSize(): int
+ operator<<(ostream&, const List<T>&): ostream&   {friend}

Node<T>
- data: T*
- nextNode: Node<T>*
+ Node(T*)
+ ~Node()
+ getData(): const T&
+ next(): Node<T>*&

Department
- teams: Team**
- size: int
+ Department(int)
+ ~Department()   {virtual}
+ addTeam(Team*): void
+ operator[](int): Team*&
+ getSize(): int
+ costToCompany(): float   {virtual}
+ getName(): string   {virtual}
+ operator<<(ostream&, Department&): ostream&   {friend}
```

Not on the published partial UML: `totalCost` and its specialization (the target; function templates are not on the UML), `~Company()`, `Company::costToCompany`, `Department::getSize`, `Department::getName`, the `{virtual}` markers (the UML marks nothing virtual), `operator<<` (not drawn; the scenario says every class has one). `head()` is written `Node<T>* const& head() const`: a

call to the UML form `const Node<T>*&` gives a compile error. The real exam's complete UML may add members like these.

What is given

- The general template in `Templates.h`: `cpp template <class T> float totalCost(List<T>& list) { return 0; }`
- The declaration of the specialization, also in `Templates.h`: `cpp template <> float totalCost<Department*>(List<Department*>& list);`
- `totalCost` is not a member of `List`, so it cannot use `headNode`. Use `list.head()` to get the first node (or `NULL`). For a node `curr`, `curr->getData()` returns the `Department*` and `curr->next()` returns the next node.
- `Company::costToCompany()` is given as `return totalCost(departments);`. In this question the company owns its departments, and `~Company()` deletes them. (Other questions in this bank may give this job to a different class.)
- `Sales`, `HR`, `Marketing` and `Development` inherit from `Department`. `costToCompany()` is virtual. It returns the number of employees in the department times a monthly amount: Sales R15000, HR R12000, Marketing R18000, Development R25000. The base `Department` returns 0.
- `operator<<` for a Department prints `<name> (cost R<cost>)` and a newline, and then every team (`Team:` and one line per employee). It already ends every line, so do not add an extra `endl` after it.
- A Department owns its teams, and a Team owns its employees.

Behaviour of `totalCost<Department*>(List<Department*>& list)`

- Return the sum of `costToCompany()` of every Department in the list.
- Skip `NULL` entries.
- An empty list returns 0.
- Do not change the list and do not delete anything.

File layout

- `Templates.h` holds the general template (fully written) and, after it, a **declaration** of the specialization (only the header line, ending in `;`).
- You write the **definition** of the specialization in `B_3.cpp`. A full specialization is an ordinary function, not a template, so it must be defined in exactly one `.cpp` file. `B_3.cpp` is compiled on its own (it is on the g++ line).
- Because every file that includes `Templates.h` sees the declaration, `main.cpp` calls your specialization and not the general template.

Rules

- Write the specialization the way it is shown in the lectures (L19): the line `template <>`, then the function with the concrete type in angle brackets after the function name. It must match the declaration in `Templates.h` exactly (return type, name, `<type>` and parameter types). Your definition starts with the same two lines as the declaration, without the `;`:

`cpp template <> float totalCost<Department*>(List<Department*>& list) {` - Write only the definition of the specialization in `B_3.cpp`. Do not write the general template again, and do not change any header. - C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`. - Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build. - Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.

Practice question based on the published HRMS scenario. The real exam's complete UML may differ.

B3-07. Template specialization: `addition<Employee, Employee>` throws (100)

For this question, write a template specialization (a version of a template written for one exact type) in the same form as the lecture example, where adding two `char` values throws an exception. Adding two employees makes no sense. Write the specialization of `addition` for two `Employee` objects in the file `B_3.cpp`, so that it throws an `Exception`.

UML (only the classes this question needs)

```
Templates.h (function templates)
+ addition<T, U>(T, U): T
+ addition<Employee, Employee>(Employee, Employee): Employee   <- implement

Exception
+ message: string
+ Exception(string)
+ what(): string
+ operator<<(ostream&, const Exception&): ostream&   {friend}

Employee
- age: int
- firstName: string
- surName: string
- idNum: string
- gender: char
+ Employee(string, string, int, char)
+ Employee(const Employee&)
+ getFirstName(): string
+ getSurName(): string
+ getIDNum(): string
+ getAge(): int
+ getGender(): char
+ setIDNum(string): void
+ extractIntFromString(string, int, int): int   {static}
```

Not on the published partial UML: `addition` and its specialization (the target; function templates are not on the UML), the `int` return type of `extractIntFromString` (the UML shows no return type), `operator<<` (not drawn; the scenario says every class has one). The real exam's complete UML may add members like these.

What is given

- The general template in `Templates.h` (from the lectures): `cpp template <class T, class U> T addition(T v1, U v2) { return v1 + v2; }`
- The declaration of the specialization, also in `Templates.h`: `cpp template <> Employee addition<Employee, Employee>(Employee v1, Employee v2);`
- `Exception` has a constructor that takes the message. `what()` returns the message. `operator<<` for an Exception prints `Exception: <message>`.

Behaviour of `addition<Employee, Employee>(Employee v1, Employee v2)`

- Do not add anything. Throw an `Exception` with the message `Cannot add two employees`.
- Throw it by value (not a pointer, not a C-string), so that `catch (Exception& e)` catches it.
- Do not print anything.

File layout

- `Templates.h` holds the general template (fully written) and, after it, a **declaration** of the specialization (only the header line, ending in `;`).
- You write the **definition** of the specialization in `B_3.cpp`. A full specialization is an ordinary function, not a template, so it must be defined in exactly one `.cpp` file. `B_3.cpp` is compiled on its own (it is on the g++ line).
- Because every file that includes `Templates.h` sees the declaration, `main.cpp` calls your specialization and not the general template.

Rules

- Write the specialization the way it is shown in the lectures (L19): the line `template <>`, then the function with the concrete types in angle brackets after the function name. It must match the declaration in `Templates.h` exactly (return type, name, `<types>` and parameter types). Your definition starts with the same two lines as the declaration, without the `;`:

`cpp template <> Employee addition<Employee, Employee>(Employee v1, Employee v2) { -` Write only the definition of the specialization in `B_3.cpp`. Do not write the general template again, and do not change any header. - C++98 only: use `NULL`. Do not use `nullptr`, `auto` or `to_string`. - Your code must compile with `-std=c++98 -Wall -Werror`. A warning stops the build. - Memory is marked. valgrind must show no errors and no leaks.

Print formats: the print formats in this question (the given `operator<<` functions and the `ExpectedOutput` block) belong to this question only. Other questions in this bank may print the same class differently.

Build and upload

1. Open a terminal in the `B_3` folder.
2. Run `make B_3`. This compiles with `g++ -g -std=c++98 -Wall -Werror` and runs the program with `valgrind`.
3. Compare your output with the `ExpectedOutput` block at the end of `main.cpp`. It must match exactly.
4. You may change `main.cpp` to test more cases. Do not change any other given file.
5. Upload only `B_3.cpp`. Every other file, `main.cpp` included, is replaced when your work is marked.

Marking: the code must compile with no warnings, print exactly the expected output and free all memory (valgrind is used). If the code does not compile, or the program stops abnormally (for example because of an uncaught exception), the mark is 0.